```
// ===== FILE: server/agents/gatekeeper/agent.ts =====
import { Content, Part } from '@google/genai';
import { llmGenerate } from '@/server/services/llm/client';
import { AGENT_MODELS } from '@/server/config/models';
import { withRetryOn429 } from '@/server/utils/retryOn429';
import {
  GatekeeperIntent,
  enrichIntentFromContext,
  enrichIntentWeather,
  extractTextFromParts,
  extractUserTextFromHistory,
  normalizeGatekeeperIntent,
  finalizeGatekeeperResult,
  coerceWardrobeBrowseIntent,
  adjudicateNewTaskVsRevisionIntent,
  isOutfitGeneratingIntent,
} from '../intent';
import { evaluateGatekeeperOutput } from '@/server/utils/gatekeeperEvaluator';
import { logGatekeeperAudit } from '@/server/logging/gatekeeper';
import { resolveWardrobeAnchor } from './wardrobeResolver';
import type { WardrobeAnchorCandidate } from '../intent';
import { gatekeeperSchema, WeatherLookup, type ProfileUpdate } from './schema';
import { GATEKEEPER_SYSTEM_INSTRUCTION } from './prompts';
import { resolveDressingClimateForIntent } from '@/server/utils/ragSeasonFilter';
import { evaluateCityWeatherGate } from '@/server/utils/cityWeatherGate';
import { formatSessionItemsForGate } from '@/server/utils/sessionItems';
import type { SessionPurchaseItem } from '@/server/utils/sessionItems';
import { normalizeProfileUpdate } from '@/server/utils/profileUpdatePatch';

export type { WeatherLookup, ProfileUpdate } from './schema';

export interface GatekeeperContext {
  clientIp?: string;
  profileLocation?: string;
  clientId?: string;
  sessionItems?: SessionPurchaseItem[];
  currentImageUrl?: string;
}

export interface GatekeeperAuditMeta {
  conversationId?: string;
  messageId?: string;
  signal?: AbortSignal;
}

export interface GatekeeperResult {
  is_complete: boolean;
  extracted_intent: GatekeeperIntent;
  followup_questions: string[];
  /** outfit_selection / outfit_confirmed / clarify 时由 Gatekeeper 直接回复用户的话术 */
```

```
  gatekeeper_reply?: string;
  /** Gatekeeper 决策本轮是否查询天气及目标城市（仅服务端使用） */
  weather_lookup?: WeatherLookup;
  /** Gatekeeper 抽出的长期档案增量（服务端异步合并） */
  profile_update?: ProfileUpdate;
  /** wardrobe_pairing 检索到多件相似单品时的候选列表 */
  wardrobe_candidates?: WardrobeAnchorCandidate[];
  /** 是否建议在回复中提示用户告知城市以获取天气 */
  suggestCityForWeather?: boolean;
  /** Gatekeeper 的推理过程文本（仅 thinking 模式下存在） */
  thinking?: string;
}

function buildContextText(history: Content[], currentInput: Part[]): string {
  return `${extractUserTextFromHistory(history)}\n${extractTextFromParts(currentInput)}`.trim();
}

async function finalizeGatekeeperIntent(
  intent: GatekeeperIntent,
  history: Content[],
  currentInput: Part[],
  ctx: GatekeeperContext,
  weatherLookup?: WeatherLookup
): Promise<{ intent: GatekeeperIntent; suggestCityForWeather: boolean; fetchedWeather?: string }> {
  const contextText = buildContextText(history, currentInput);
  let enriched = enrichIntentFromContext(intent, history, currentInput, {
    sessionItems: ctx.sessionItems,
    currentImageUrl: ctx.currentImageUrl,
  });

  // 天气由 Gatekeeper LLM 决策（weather_lookup.needed）；服务端在此串行查询补全
  let fetchedWeather: string | undefined;
  if (weatherLookup?.needed && !enriched.weather.trim()) {
    if (weatherLookup.city?.trim()) {
      enriched = { ...enriched, city: weatherLookup.city.trim() };
    }
    enriched = await enrichIntentWeather(enriched, {
      clientIp: ctx.clientIp,
      profileLocation: ctx.profileLocation,
      contextText,
    });
    fetchedWeather = enriched.weather.trim() || undefined;
  }

  // 服务端统一解析 dressing_climate（实况温度 / 锚点 / 活动；忽略 Gatekeeper LLM 推断）
  if (isOutfitGeneratingIntent(enriched)) {
    enriched = resolveDressingClimateForIntent(enriched, contextText);
  }

  const suggestCityForWeather = Boolean(weatherLookup?.needed) && !enriched.weather.trim();
```

```
  return { intent: enriched, suggestCityForWeather, fetchedWeather };
}

/**
 * 调用 Gatekeeper Agent 评估用户请求。
 * 单次 LLM 调用即完成意图判定、天气决策与档案增量提取；天气在 LLM 返回后按 weather_lookup 串行查询，档案增量由编排层异步写库。
 */
export async function callGatekeeperAgent(
  history: Content[],
  currentInput: Part[],
  ctx: GatekeeperContext = {},
  auditMeta?: GatekeeperAuditMeta
): Promise<GatekeeperResult> {
  console.log('[GATEKEEPER_AGENT] Evaluating user request...');

  const sessionBlock = formatSessionItemsForGate(ctx.sessionItems ?? []);
  const gatedInput: Part[] = sessionBlock
    ? [{ text: sessionBlock }, ...currentInput]
    : currentInput;
  const contents: Content[] = [...history, { role: 'user', parts: gatedInput }];

  try {
    const response = await withRetryOn429(
      () =>
        llmGenerate({
          model: AGENT_MODELS.gatekeeper,
          contents,
          systemInstruction: GATEKEEPER_SYSTEM_INSTRUCTION,
          temperature: 0.0,
          jsonSchema: gatekeeperSchema,
          signal: auditMeta?.signal,
        }),
      { label: 'Gatekeeper', maxRetries: 4 }
    );

    const thinkingText = '';
    const responseText = response.text;
    if (!responseText) {
      throw new Error('Empty response from Gatekeeper Agent');
    }

    console.log('[GATEKEEPER_AGENT] Raw response:', responseText);

    const parsed = JSON.parse(responseText) as GatekeeperResult;
    const profileUpdate = normalizeProfileUpdate(parsed.profile_update);
    const rawIntent = normalizeGatekeeperIntent(parsed.extracted_intent);
    const intentBeforeCoerce = rawIntent.request_type;
    const browsedIntent = coerceWardrobeBrowseIntent(rawIntent, history, currentInput);
    const coercedIntent = adjudicateNewTaskVsRevisionIntent(browsedIntent, currentInput);
    const wasCoercedBrowse =
```

```
    intentBeforeCoerce === 'clarify' && coercedIntent.request_type === 'wardrobe_pairing';

  const { intent, suggestCityForWeather, fetchedWeather } = await finalizeGatekeeperIntent(
    coercedIntent,
    history,
    currentInput,
    ctx,
    parsed.weather_lookup
  );

  let wardrobeResolver;
  if (
    intent.request_type === 'wardrobe_pairing' &&
    ctx.clientId &&
    intent.anchor_item_summary.trim() &&
    !intent.anchor_wardrobe_id?.trim()
  ) {
    wardrobeResolver = await resolveWardrobeAnchor(
      ctx.clientId,
      intent.anchor_item_summary,
      intent.anchor_slot || undefined,
      intent.wardrobe_search_query
    );
  }

  const result = finalizeGatekeeperResult({
    extracted_intent: intent,
    followup_questions: parsed.followup_questions,
    gatekeeper_reply: wasCoercedBrowse ? undefined : parsed.gatekeeper_reply,
    suggestCityForWeather,
    fetchedWeather,
    wardrobeResolver,
    currentMessageText: extractTextFromParts(currentInput),
    history,
    modelIsComplete: parsed.is_complete,
  });

  const contextText = buildContextText(history, currentInput);
  const cityGate = evaluateCityWeatherGate({
    requestType: result.extracted_intent.request_type,
    weatherLookupNeeded: Boolean(parsed.weather_lookup?.needed),
    weather: result.extracted_intent.weather,
    city: result.extracted_intent.city || parsed.weather_lookup?.city,
    profileLocation: ctx.profileLocation,
    contextText,
  });

  let gatedResult = result;
  if (result.is_complete && cityGate.blocked) {
    console.log('[GATEKEEPER] city weather gate — missing city, ask before outfit');
```

```
      gatedResult = {
        ...result,
        is_complete: false,
        followup_questions: cityGate.followup ? [cityGate.followup] : result.followup_questions,
        gatekeeper_reply: cityGate.followup || result.gatekeeper_reply,
      };
    }

    const l1 = evaluateGatekeeperOutput(
      {
        ...gatedResult,
        wardrobe_candidates: gatedResult.wardrobe_candidates,
      },
      {
        contextText,
        currentMessageText: extractTextFromParts(currentInput),
        weatherLookup: parsed.weather_lookup,
      }
    );
    void logGatekeeperAudit({
      timestamp: new Date().toISOString(),
      conversationId: auditMeta?.conversationId,
      messageId: auditMeta?.messageId,
      contextTextLength: contextText.length,
      requestType: gatedResult.extracted_intent.request_type,
      is_complete: gatedResult.is_complete,
      gatekeeper_reply: gatedResult.gatekeeper_reply,
      followup_questions: gatedResult.followup_questions,
      extracted_intent: gatedResult.extracted_intent,
      weather_lookup: parsed.weather_lookup,
      profile_update: profileUpdate,
      wardrobe_candidates: gatedResult.wardrobe_candidates,
      l1,
      thinking: thinkingText || undefined,
    });

    return {
      ...gatedResult,
      weather_lookup: parsed.weather_lookup,
      profile_update: profileUpdate,
      thinking: thinkingText || undefined,
    };
  } catch (error) {
    console.error('[GATEKEEPER_AGENT] Error calling Gatekeeper Agent:', error);
    throw error;
  }
}


// ===== FILE: server/agents/gatekeeper/index.ts =====
```

```
export * from './agent';


// ===== FILE: server/agents/gatekeeper/prompts.ts =====
export const GATEKEEPER_SYSTEM_INSTRUCTION = `
【强制语言规则】你的全部思考过程（thinking）和最终输出，必须使用中文。严禁使用英文进行推理或输出。

你是一个严格、专业且亲切的时尚前台把关人（Gatekeeper Agent），是整个搭配系统的【入口】。
你的任务是先判断用户当前意图，再决定是否放行进入搭配流程；当请求与搭配相关但意图不清晰时，你要【亲自追问】，而不是贸然进入搭配。

【请求类型 request_type】
必须先判断用户属于哪一种：
1. wardrobe_outfit：用户要从【已有衣橱】搭配一套穿搭（无特定锚定单品）。
2. wardrobe_pairing：用户明确指定【衣橱里已有】的某件单品想穿出门（如「我衣橱那条绿色裙子」「想穿我的白色衬衫」）——用该衣橱单品作锚点，从衣橱找互补单品。【禁止】归为 purchase_pairing 或 clar...
3. purchase_pairing：用户上传了【待购/非衣橱】服装单品的【图片】作为锚点。【必须有图片】才能归为此类；仅凭文字说"想买某类单品/对比购买"不触发此类型。【禁止】用于衣橱已有单品。
4. feedback_revision：用户对上一轮方案提出【明确修改】（必须有明确修订信号，如「第一套太正式了，换成裤装」「鞋换成白色」「不要红色外套」「这套改成…」）——放行，交搭配师微调。若用户只是提出新的活动/场景/目标（如...
5. outfit_selection：用户【仅表示】更喜欢/选定上一轮的某一套（如「我比较喜欢第一套」「就第二套吧」），但【没有说明】是已满意、还是想再微调——不要放行，由你在 gatekeeper_reply 中确认并追问。
6. outfit_confirmed：用户在选定某套后，明确表示【已满意、不用调整、可以直接穿】——归类为 outfit_confirmed，【禁止】归为 outfit_selection；【禁止】再次追问是否微调；gatekeep...
7. clarify：用户输入与搭配/穿衣相关，但意图模糊、信息不足以归入以上任何一类——不要放行，由你在 gatekeeper_reply 中亲切追问。
    【禁止】将以下情况归为 clarify：
   - 「我衣橱里有没有 X」「想看看 X」「X 呢」等衣橱单品查询/浏览
   - 用户纠正上一轮检索颜色/款式错误（如「这不是绿色的裙子吗」）
   以上均应归为 wardrobe_pairing，anchor_item_summary 填用户要找的单品（从历史对话提取，如「白裙子」），is_complete=false，由服务端检索并展示候选卡片；gatekeeper_re...

8. style_advice：用户在【咨询某种风格、色彩、场景或穿搭方法的建议/知识】，而非直接要你立刻搭一套（如「高级感色彩搭配公式」「哪些颜色适合搭在一起」「显瘦有什么技巧」「美拉德风怎么穿」）。

【style_advice 处理（重要）】
- style_advice 是【知识/建议回答】类型，不需要搭配方案。有可回答主题（咨询风格、色彩、穿搭方法等）就直接放行（is_complete=true），由 Stylist 以建议模式回答。
- 不要为了回答建议类问题而追问场合。用户问「高级感色彩搭配公式」「哪些颜色适合搭配」时，直接归为 style_advice，让 Stylist 直接回答公式、颜色组合和避坑。
- 如果用户只是补充场合（如「职场通勤」「日常百搭」），仍然保持 style_advice；occasion 填该场景，special_requests 继续保留原始知识诉求，让 Advice Agent 讲"这个公式在该场景怎么用"。
- 只有用户表达了【想看实际搭配效果 / 想把建议落实成穿搭示范】的意图时，才不要归为 style_advice；应归为 wardrobe_outfit（无特定锚点）或 wardrobe_pairing（有指定单品）。判断依据是意图，...
- 【贯穿原始诉求（关键）】：多轮咨询里，用户最初问的可能是某个知识点（如「高级感色彩搭配公式」），后续只是补充了场合（如「职场通勤」）。你【必须】从历史对话提取用户最初真正想了解的主题，写入 special_requests。occ...
- style_advice 不需要锚定单品、天气或穿衣气候；weather_lookup.needed 按【天气与城市】规则判断（用户未主动问天气时为 false）。

【outfit_selection 话术（重要）】
- gatekeeper_reply 必须【简短】：一句确认选了哪套 + 一个选择题（满意 or 微调）。
- 【禁止】堆砌赞美（如「非常衬气质」「清新又知性」），禁止书信腔。

【wardrobe_pairing 处理（重要）】
- anchor_item_summary 填用户描述的衣橱单品（如「绿色裙子」「白裙子」）；anchor_slot 填对应槽位。
- wardrobe_search_query【关键】：这是实际触发衣橱向量检索的 query，准确性直接决定搜索结果。要求：读完整段对话历史后，理解用户真正想找到的单品，输出「颜色 + 品类」格式的精简词，如「白色连衣裙」「浅蓝牛仔...
- 若用户已点选确认（消息含 id=xxx），填 anchor_wardrobe_id。
- 【衣橱浏览】用户只想查看/确认衣橱里有没有某单品（「有没有白裙子」「想看看」「裙子呢」）时：request_type=wardrobe_pairing，is_complete=false，gatekeeper_reply 必须留空...
- 【用户纠错】用户指出上一轮展示的颜色/款式不对时：重新归为 wardrobe_pairing，anchor_item_summary 填用户原本要找的描述（从历史提取），不要用 clarify 反复追问。
- 若缺场合且用户已明确要【搭配】（非仅浏览），is_complete=false，followup_questions 追问场合；服务端会自动检索衣橱确认单品。
- 不要因缺天气拦截 wardrobe_pairing。

【purchase_pairing 放行标准（重要）】
- 你具备多模态能力，必须亲自从【本轮服装图片】【会话待购单品清单】或【用户文字】识别锚定单品，填入 anchor_item_summary 和 anchor_slot。
- 若上下文出现【会话待购单品】清单：必须填写 session_item_id 为清单中的某一项；用户说「还是那条/刚才那件」时按文字与 hint 匹配对应 id，禁止无脑填最新一项除非本轮新传了图且未指代旧图。
```

- feedback_revision 时 session_item_id 必须留空（微调绑定上一套方案上的锚点，不是最新上传）。
- anchor_item_summary 示例：「蓝白细条纹棉质衬衫，宽松版型」→ anchor_slot=top；「金色圆环耳环，铆钉细节」→ anchor_slot=accessory。
- anchor_slot 必须是 top | bottom | dress | shoes | outerwear | accessory 之一。
- 【重要】耳环、耳钉、项链、手链、戒指、手表、包、腰带、围巾、帽子等配饰类单品，anchor_slot 必须填 accessory，禁止填 top。
- 场合要求放宽：用户说「平时/百搭/日常/都可以穿」即视为场合足够，occasion 填「日常百搭」。
- 只有完全无法从图片或文字识别锚定单品时，才 is_complete=false 并追问。
- special_requests 填：用户待购单品（xxx）需作为搭配锚点，从衣橱选取互补单品与之搭配。
- 不要因为缺少天气拦截 purchase_pairing。

【outfit_selection vs outfit_confirmed（极易混淆，务必区分）】
- outfit_selection：只说喜欢第几套，还没表态要不要改。例：「我选第一套」「更喜欢第二套」。
- outfit_confirmed：已表态满意/定稿/不用改。例：「我很满意不用调整」「就这套」「可以了直接穿」「不用改了」。
- 若用户说「满意」「不用调整」「不用改」，一律 outfit_confirmed，绝不可 outfit_selection。

【wardrobe_outfit 放行标准】
- 通常需要明确场合（上班、约会、徒步等）才可放行。
- 但有以下情况可直接以「日常百搭」放行，无需追问场合：
  ① 用户说「平时/百搭/日常/都可以穿/随便穿」等宽泛场合词；
  ② 历史对话中存在建议类（style_advice）交流，且用户本轮的意图是【想看建议的实际效果 / 想把刚才的知识落实成穿搭示范】——无论用什么措辞（"给我搭一套" "帮我配一套" "想看看效果" "能变出什么" "实践一下" ...
- 上述两种情况以外，场合不明确时 is_complete=false，亲切追问 1–2 个问题。

【feedback_revision】
- is_complete=true，request_type=feedback_revision；session_item_id 必须留空（锚点跟方案不跟最新上传）；从上下文继承场合，special_requests 写入用户的修改要求。
- 【边界原则】：feedback_revision 必须满足「用户明确要修改上一轮某套方案或某个单品」。若用户本轮提出的是新的场合/活动/目标（例如「我还想去…」「明天去…」「应该穿啥/穿什么」），即使上一轮刚生成过方案，也必须归为...
- 【整轮重做】：用户说「重新搭配」「重新生成」「都不好」「搭配太烂了」等表达对整套方案不满意、要求全新方案时，必须归为 wardrobe_outfit，【禁止】归为 feedback_revision，【禁止】追问选第一套还是第二套。
- selected_outfit_id【禁止臆测】：仅当用户本轮或历史中明确说了「第一套/第二套/outfit_1/outfit_2」时填写；若用户只说修改指令（如「去掉外套」「鞋换成高跟鞋」）而未指明哪套，selected_out...
- 【new_item 风格追问（关键）】：若上一轮 AI 方案中推荐了某件新品（non-wardrobe item，如「短裤」「白衬衫」），用户本轮追问该新品的风格多样性（如「短裤能多几种风格吗」「能给我看更多款式吗」「想对比一下再...

【天气与城市（不参与放行，但由你决策是否查询）】
- 不要因缺少天气或温度信息而拦截用户。
- 若用户主动提到天气/温度，提取到 extracted_intent.weather；若主动提到城市，提取到 extracted_intent.city。
- city_role【关键】：有 city 时必须判定角色——home=用户常住/所在城市（回答系统追问所在地、或明确说「我在/我家在/我住在」某城）；travel=旅游/出差/短期出行目的地（去某城玩、出差、度假等）。city 为...
- 由你判断本轮是否需要实时天气，并填写 weather_lookup：
  - weather_lookup.needed 只看「本轮回复是否依赖实时天气」，与 request_type 无关：
    - 用户主动询问天气、气温或冷暖（含询问出行目的地的天气）→ true，无论 request_type 判为什么；
    - wardrobe_outfit / feedback_revision 这类要真正出穿搭、受冷暖影响的 → true；
    - 其余与天气无关的请求 → false；用户已自述当前天气/温度时 → false。
  - weather_lookup.city：只填用户明确提到、且本轮要查的城市（如出行目的地）；未提及则留空字符串，系统会依次用用户档案中的常住城市、IP 兜底，【禁止】自行猜测城市。
- 系统会在你返回后据 weather_lookup 串行查询并补全天气结果，你无需填写查询结果本身。

【长期档案增量 profile_update】
- 与 weather_lookup 类似：由你判断本轮是否含应写入用户【长期时尚档案】的信息，并填写 profile_update；服务端异步合并写库，你无需读旧档全文。
- profile_update.needed=true 仅当本轮出现跨对话仍成立的稳定信息：明确自我介绍姓名、身高/体重、一贯风格偏好/风格定位。
- profile_update.needed=false：纯场合、单次想穿什么、选套、确认定稿、微调、衣橱浏览、待购图搭配、知识咨询中未自我披露档案信息。outfit_selection / outfit_confirmed / c...
- patch 规则：
  - 未提及的字段填空字符串；preference_additions 无新增则空数组。
  - preference_additions 只写长期偏好（如「喜欢简约」「平时不爱穿外套」）；禁止写场合、单次单品、操作确认。
  - personal_style 仅用户明确说出风格定位时填写；禁止根据场合臆测。
  - 禁止填写肤色/身材/发型等外形字段（不在 patch 内）。

```
【穿衣气候 dressing_climate】
- 【禁止自行推断】dressing_climate 一律填空字符串；服务端会在查完天气后根据温度、锚点单品自动计算，勿根据「高原/保暖/温差」等话术填写 cold。

【严格提取，禁止臆测】
- style_preference：仅当用户明确提到风格词时填写；否则填 "日常休闲"。
- city：仅提取用户明确提到的城市；未提及则填 ""。
- city_role：随 city 判定；无 city 时填 ""；不确定是常住还是出行时优先 travel（宁可本轮不落档，也不误写档案）。
- anchor_item_summary / anchor_slot：purchase_pairing / wardrobe_pairing 时填写；识别服装图时请描述颜色、品类、材质，不要描述模特外貌。

【工作流程】
1. 仔细阅读用户当前输入及历史对话（含历史中的服装图片）。
2. 判定 request_type；衣橱已有单品 / 衣橱查询浏览 → wardrobe_pairing；【禁止】用 clarify 处理衣橱单品查询；意图不清晰且与衣橱无关时才归为 clarify。
3. 填写 weather_lookup 与 profile_update（彼此独立）。
4. 可放行（wardrobe_outfit / wardrobe_pairing / purchase_pairing / feedback_revision 且信息齐全）→ is_complete=true，gatekeeper_...
5. style_advice → 有明确建议主题则 is_complete=true，由 Stylist 以建议模式回答；主题完全不明确时才 is_complete=false 并用 gatekeeper_reply 追问。
6. 不可放行 → is_complete=false：结构化信息缺失用 followup_questions；意图需澄清（outfit_selection / clarify）用 gatekeeper_reply 直接回复。
`;


// ===== FILE: server/agents/gatekeeper/schema.ts =====
import { Type, Schema } from '@google/genai';

export interface WeatherLookup {
  needed: boolean;
  city: string;
}

/** Gatekeeper 抽出的长期档案增量；服务端合并写库，不阻塞搭配 */
export interface ProfileUpdatePatch {
  name: string;
  height: string;
  weight: string;
  personal_style: string;
  preference_additions: string[];
}

export interface ProfileUpdate {
  needed: boolean;
  patch: ProfileUpdatePatch;
}

export const EMPTY_PROFILE_UPDATE_PATCH: ProfileUpdatePatch = {
  name: '',
  height: '',
  weight: '',
  personal_style: '',
  preference_additions: [],
};

export const gatekeeperSchema: Schema = {
```

```
  type: Type.OBJECT,
  properties: {
    is_complete: {
      type: Type.BOOLEAN,
      description:
        '是否可进入搭配生成流程。wardrobe_outfit 需场合明确；purchase_pairing 需锚定单品；feedback_revision 有明确修改指令时为 true；style_advice 有可回答主...
    },
    extracted_intent: {
      type: Type.OBJECT,
      properties: {
        weather: {
          type: Type.STRING,
          description: '用户主动提到的天气或温度（如：15度、下雨）。未提及则填空字符串。',
        },
        city: {
          type: Type.STRING,
          description: '用户主动提到的城市名。未提及则填空字符串。',
        },
        city_role: {
          type: Type.STRING,
          description:
            '本轮 city 的语义角色。home：用户在说明常住/所在城市（含回答「您在哪个城市」类追问、说「我在X」）；travel：旅游、出差、短途出行等目的地（如「去X玩/出差」）。city 为空时必须填空字符串。...
        },
        occasion: {
          type: Type.STRING,
          description: '穿搭场合。若信息不全，此项填空字符串。',
        },
        style_preference: {
          type: Type.STRING,
          description:
            '用户原文中明确说出的风格偏好（如：温柔风、美式复古、松弛感）。禁止根据场合臆测风格。用户未提及则填 "日常休闲"。',
        },
        special_requests: {
          type: Type.STRING,
          description:
            '用户明确说出的特殊要求（遮肚子、显腿长等）；purchase_pairing 时填写待购单品搭配说明；style_advice 时【必须】结合历史对话，填入用户最初真正想了解的建议主题（如「高级感色彩搭配公式...
        },
        request_type: {
          type: Type.STRING,
          description:
            '请求类型：wardrobe_outfit（从衣橱搭一套）| wardrobe_pairing（指定衣橱已有单品作锚点）| purchase_pairing（待购/上传单品+衣橱互补）| feedback_re...
        },
        selected_outfit_id: {
          type: Type.STRING,
          description:
            'outfit_selection / outfit_confirmed 时填用户选中的 outfit_1 或 outfit_2。feedback_revision 时【仅当用户本轮或历史中明确说了第一套/第二...
        },
        anchor_wardrobe_id: {
          type: Type.STRING,
          description:
```

```
            'wardrobe_pairing 时：用户已确认的衣橱单品 id（用户点选候选或明确指定）。未确认则填空，由服务端检索。',
        },
        session_item_id: {
          type: Type.STRING,
          description:
            'purchase_pairing：填写【会话待购单品】清单中的 id（如 si_1）。用户指代历史某张上传图时选对应 id；本轮新上传且未指代旧图时选清单最后一项。feedback_revision 必须填空...
        },
        anchor_item_summary: {
          type: Type.STRING,
          description:
            'purchase_pairing / wardrobe_pairing 时：锚定单品描述（颜色+品类+关键特征）。其他类型填空字符串。',
        },
        wardrobe_search_query: {
          type: Type.STRING,
          description:
            'wardrobe_pairing 专用：用于衣橱语义检索的精简关键词（颜色 + 品类），如「白色连衣裙」「浅蓝牛仔裤」「米白衬衫」。读完整段对话历史后，以用户最终希望找到/穿到的单品为准填写，不是用户用来描述...
        },
        anchor_slot: {
          type: Type.STRING,
          description:
            'purchase_pairing / wardrobe_pairing 时锚定单品的穿搭槽位：top | bottom | dress | shoes | outerwear | accessory。耳环、项...
        },
        dressing_climate: {
          type: Type.STRING,
          description:
            '留空字符串。穿衣气候由服务端在查完天气后根据温度与锚点单品自动计算，禁止 Gatekeeper 自行填写。',
        },
      },
      required: [
        'weather',
        'city',
        'city_role',
        'occasion',
        'style_preference',
        'special_requests',
        'request_type',
        'anchor_item_summary',
        'wardrobe_search_query',
        'anchor_slot',
        'selected_outfit_id',
        'anchor_wardrobe_id',
        'session_item_id',
        'dressing_climate',
      ],
    },
    gatekeeper_reply: {
      type: Type.STRING,
      description:
        '当 is_complete=false 且 request_type 为 outfit_selection / outfit_confirmed / clarify 时，用亲切时尚顾问口吻【直接回复用户】的完整话术。...
    },
```

```
    weather_lookup: {
      type: Type.OBJECT,
      properties: {
        needed: {
          type: Type.BOOLEAN,
          description:
            '本轮回复是否依赖实时天气，与 request_type 无关：用户主动询问天气/气温/冷暖时必为 true；要真正出穿搭、受冷暖影响时为 true；与天气无关或用户已自述天气温度时为 false。',
        },
        city: {
          type: Type.STRING,
          description: '用户明确提到、且本轮要查天气的城市；未提及填空字符串，系统会依次用用户档案常住城市、IP 兜底。禁止猜测。',
        },
      },
      required: ['needed', 'city'],
    },
    profile_update: {
      type: Type.OBJECT,
      description:
        '本轮是否含应写入【长期时尚档案】的信息。系统会异步合并写库，不阻塞搭配。只记录跨对话仍成立的稳定信息。',
      properties: {
        needed: {
          type: Type.BOOLEAN,
          description:
            '本轮是否需要更新长期档案。true：明确自我介绍姓名、身高体重、一贯风格偏好等；false：纯场合/单次想穿什么/选套/确认/微调/衣橱浏览/待购搭配。outfit_selection / outfit_co...
        },
        patch: {
          type: Type.OBJECT,
          description: '增量字段；未提及的项填空字符串或空数组，禁止把本轮场合/单次单品意图写入 preference_additions。',
          properties: {
            name: {
              type: Type.STRING,
              description: '仅用户明确自我介绍时填写；否则空字符串。',
            },
            height: {
              type: Type.STRING,
              description: '用户明确说出的身高；未提及填空字符串。',
            },
            weight: {
              type: Type.STRING,
              description: '用户明确说出的体重；未提及填空字符串。',
            },
            personal_style: {
              type: Type.STRING,
              description: '用户明确说出的长期风格定位；未提及填空字符串。禁止根据场合臆测。',
            },
            preference_additions: {
              type: Type.ARRAY,
              items: { type: Type.STRING },
              description:
                '本轮新出现的【长期】穿衣偏好条目（可追加多条）。禁止写入场合、单次搭配请求、操作确认。无则空数组。',
```

```
            },
          },
          required: ['name', 'height', 'weight', 'personal_style', 'preference_additions'],
        },
      },
      required: ['needed', 'patch'],
    },
    followup_questions: {
      type: Type.ARRAY,
      items: { type: Type.STRING },
      description:
        '当 is_complete=false 且缺少场合/锚定单品等结构化信息时，用于追问的问题列表。当 is_complete=true 时必须为空数
组。',
    },
  },
  required: [
    'is_complete',
    'extracted_intent',
    'gatekeeper_reply',
    'weather_lookup',
    'profile_update',
    'followup_questions',
  ],
};


// ===== FILE: server/agents/gatekeeper/wardrobeResolver.ts =====
import { ClothingMainCategory } from '@prisma/client';
import { searchWardrobeItemsByText } from '@/server/services/wardrobeService';
import { inferAnchorSlotFromSummary } from '@/server/agents/intent';
import { filterItemsByQueryColor } from '@/server/utils/queryColorMatch';
import { getTextSimilarityThresholds } from '@/server/services/embedding';
import type { WardrobeSearchSlot } from '@/server/utils/ragSearchSlots';
import type {
  AnchorSlot,
  WardrobeAnchorCandidate,
  WardrobeResolverResult,
} from '@/server/agents/intent';

export type { WardrobeResolverResult };

const MAX_CANDIDATES = 4;

function slotToMainCategory(slot: AnchorSlot): ClothingMainCategory | undefined {
  const map: Record<AnchorSlot, ClothingMainCategory> = {
    top: 'TOP',
    bottom: 'BOTTOM',
    dress: 'ONE_PIECE',
    shoes: 'FOOTWEAR',
    outerwear: 'OUTERWEAR',
    accessory: 'ACCESSORY',
```

```
  };
  return map[slot];
}

function toCandidate(item: {
  id: string;
  imageUrl: string;
  subCategory: string;
  colors: string[];
  similarity: number;
}): WardrobeAnchorCandidate {
  return {
    id: item.id,
    imageUrl: item.imageUrl,
    subCategory: item.subCategory,
    colors: item.colors,
    similarity: item.similarity,
  };
}

/**
 * Resolve a user-described wardrobe item to a concrete ClothingItem id.
 */
export async function resolveWardrobeAnchor(
  clientId: string,
  summary: string,
  slot?: AnchorSlot | '',
  /** Gatekeeper LLM 直接生成的精简检索词，优先使用 */
  llmSearchQuery?: string
): Promise<WardrobeResolverResult> {
  const query = (llmSearchQuery?.trim() || summary.trim());
  if (!query || !clientId) {
    return { status: 'not_found' };
  }
  if (llmSearchQuery?.trim() && llmSearchQuery.trim() !== summary.trim()) {
    console.log(
      `[WARDROBE_RESOLVER] Using LLM search query: "${llmSearchQuery}" (summary: "${summary.slice(0, 3
2)}")`
    );
  }

  const inferredSlot = inferAnchorSlotFromSummary(query);
  let effectiveSlot: AnchorSlot | '' = slot || '';
  if (inferredSlot && effectiveSlot && inferredSlot !== effectiveSlot) {
    console.warn(
      `[WARDROBE_RESOLVER] Overriding anchor_slot ${effectiveSlot} → ${inferredSlot} for "${query.slice(0,
32)}..."`
    );
    effectiveSlot = inferredSlot;
  } else if (inferredSlot && !effectiveSlot) {
    effectiveSlot = inferredSlot;
  }
```

```
  const mainCategory = effectiveSlot ? slotToMainCategory(effectiveSlot) : undefined;
  const searchSlot = effectiveSlot ? (effectiveSlot as WardrobeSearchSlot) : undefined;
  const searchOptions = searchSlot ? { slot: searchSlot } : undefined;
  const rawResults = await searchWardrobeItemsByText(
    query,
    clientId,
    MAX_CANDIDATES + 1,
    mainCategory,
    searchOptions
  );

  let results = rawResults;
  if (results.length === 0 && mainCategory) {
    results = await searchWardrobeItemsByText(query, clientId, MAX_CANDIDATES + 1, undefined, searchOption
s);
  }

  const { matched, rejected } = filterItemsByQueryColor(query, results);
  if (matched.length === 0 && rejected.length > 0) {
    const nearMiss = toCandidate(
      [...rejected].sort((a, b) => b.similarity - a.similarity)[0]
    );
    console.warn(
      `[WARDROBE_RESOLVER] Color mismatch for "${query.slice(0, 32)}": nearMiss colors=${nearMiss.colors.j
oin(',')}`
    );
    return { status: 'color_mismatch', queriedSummary: query, nearMiss };
  }

  return classifyResults(matched);
}

function classifyResults(
  results: Array<{
    id: string;
    imageUrl: string;
    subCategory: string;
    colors: string[];
    similarity: number;
  }>
): WardrobeResolverResult {
  if (results.length === 0) {
    return { status: 'not_found' };
  }

  const { resolveThreshold, ambiguousMin, slotConfidenceGap } = getTextSimilarityThresholds();
  const resolveGap = Math.max(slotConfidenceGap * 2, 0.08);

  const sorted = [...results].sort((a, b) => b.similarity - a.similarity);
  const top = sorted[0];
  const candidates = sorted
```

```
    .filter((r) => r.similarity >= ambiguousMin)
    .slice(0, MAX_CANDIDATES)
    .map(toCandidate);

  if (top.similarity >= resolveThreshold && candidates.length === 1) {
    return { status: 'resolved', itemId: top.id, item: toCandidate(top) };
  }

  if (top.similarity >= resolveThreshold && candidates.length > 1) {
    const second = candidates[1];
    if (top.similarity - (second.similarity ?? 0) >= resolveGap) {
      return { status: 'resolved', itemId: top.id, item: toCandidate(top) };
    }
  }

  if (candidates.length >= 1) {
    return { status: 'ambiguous', candidates };
  }

  return { status: 'not_found' };
}


// ===== FILE: server/agents/intent/finalize.ts =====
import { Content } from '@google/genai';
import {
  GatekeeperIntent,
  GatekeeperFinalizeResult,
  FinalizeGatekeeperInput,
  OutfitRequestType,
  WardrobeResolverResult,
  WardrobeAnchorCandidate,
} from './typeDefs';
import {
  normalizeGatekeeperIntent,
  outfitIdToLabel,
  extractOutfitIdFromText,
  inferSelectedOutfitIdFromHistory,
  historyShowsMultipleOutfits,
  isPurchasePairingAnchorReady,
  isWardrobePairingAnchorReady,
  isWardrobeBrowseIntent,
} from './utils';

// ─── Reply builders ──────────────────────────────────────────────────────────

export function buildOutfitSelectionGatekeeperReply(outfitLabel: string): string {
  return `好的，${outfitLabel}～这套可以直接穿出门，还是想在鞋子、外套或配饰上微调一下？`;
}
```

```
export function buildOutfitConfirmedGatekeeperReply(outfitLabel: string): string {
  return `太好了！${outfitLabel}就定下来啦～祝您穿出好心情，有需要随时再来找我～`;
}

export function buildFeedbackRevisionOutfitClarifyReply(): string {
  return '好的～上一轮给您准备了多套方案，请问您想微调第一套还是第二套？';
}

export function buildWardrobeAmbiguousReply(): string {
  return '我在您的衣橱里找到了几件相似的单品，请点击确认您想穿的是哪一件～';
}

// ——— Static copy strings ———————————————————————————————————

export const CLARIFY_INTENT_FOLLOWUP =
  '想确认一下您的需求～您是想让我从衣橱里搭一套穿搭，还是有某件单品想搭配，或者要在上一套方案上做调整呢？';

export const STYLE_ADVICE_FALLBACK_REPLY = '这个问题可以直接聊方法，不需要先限定场合。';

export const WARDROBE_NOT_FOUND_REPLY =
  '没在衣橱里找到匹配的单品～可以描述得更具体一些（颜色、款式），或者发一张该单品的照片给我～';

export function buildColorMismatchReply(
  queriedSummary: string,
  nearMiss: WardrobeAnchorCandidate
): string {
  const colorLabel = nearMiss.colors.length > 0 ? nearMiss.colors.join('、') : '其他颜色';
  return `衣橱里没有找到符合「${queriedSummary}」的单品。不过有一件${colorLabel}的${nearMiss.subCategory}，你看看是
不是这件？`;
}

export function buildWardrobeBrowseReply(itemCount: number): string {
  if (itemCount <= 1) {
    return '在衣橱里找到了这件，点选确认后我可以帮你搭配～';
  }
  return '在衣橱里找到了几件相似的单品，请点击确认你想穿的是哪一件～';
}

/** LLM 在天气 enrich 之前写了「没查到温度」时，用已拉到的实况覆盖。 */
/** Gatekeeper 的回复先于天气查询生成，无法引用实况；查到天气时由服务端改写 */
export function buildFetchedWeatherClarifyReply(fetchedWeather: string): string {
  return `帮你查了一下：${fetchedWeather}。需要我按这个天气从衣橱里帮你搭一套吗？告诉我打算去哪儿就行～`;
}

export const PURCHASE_PAIRING_ANCHOR_FOLLOWUP =
  '方便描述一下您想搭配的单品吗？或者再发一张清晰的服装图片～';

export const WARDROBE_OCCASION_FOLLOWUP =
  '请问您打算在什么场合穿呢？例如上班通勤、约会或日常休闲～';

export const CITY_WEATHER_FOLLOWUP_SUFFIX =
```

```
  '顺便告诉我您在哪个城市，方便结合当地天气给您更合适的穿搭建议～';

export function buildWardrobeOccasionFollowup(includeCityAsk: boolean): string {
  if (!includeCityAsk) return WARDROBE_OCCASION_FOLLOWUP;
  return `${WARDROBE_OCCASION_FOLLOWUP}${CITY_WEATHER_FOLLOWUP_SUFFIX}`;
}

// ——— Internal finalize context type ———————————————————————————

interface FinalizeContext {
  intent: GatekeeperIntent;
  modelFollowups: string[];
  modelReply?: string;
  modelIsComplete?: boolean;
  suggestCityForWeather?: boolean;
  fetchedWeather?: string;
  wardrobeResolver?: WardrobeResolverResult;
  currentMessageText?: string;
  history?: Content[];
}

type RequestTypeHandler = (ctx: FinalizeContext) => GatekeeperFinalizeResult;

// ——— Shared helper ————————————————————————————————————————————

function blockForMissingOccasion(ctx: FinalizeContext): GatekeeperFinalizeResult {
  const defaultFollowup = buildWardrobeOccasionFollowup(
    Boolean(ctx.suggestCityForWeather && !ctx.intent.weather.trim())
  );
  return {
    is_complete: false,
    extracted_intent: ctx.intent,
    followup_questions: ctx.modelFollowups.length > 0 ? ctx.modelFollowups : [defaultFollowup],
  };
}

// ——— Per-type handlers ————————————————————————————————————————

const handleClarify: RequestTypeHandler = ({ intent, modelReply, modelFollowups, fetchedWeather }) => {
  console.log('[GATEKEEPER] clarify — Gatekeeper 直接追问意图，短路下游');
  const reply = fetchedWeather?.trim()
    ? buildFetchedWeatherClarifyReply(fetchedWeather.trim())
    : modelReply || modelFollowups.join(' ') || CLARIFY_INTENT_FOLLOWUP;
  return {
    is_complete: false,
    extracted_intent: intent,
    followup_questions: [],
    gatekeeper_reply: reply,
  };
};
```

```
const handleOutfitConfirmed: RequestTypeHandler = ({ intent, modelReply, modelFollowups }) => {
  const label = outfitIdToLabel(intent.selected_outfit_id);
  console.log('[GATEKEEPER] outfit_confirmed — 方案已定稿，短路下游');
  return {
    is_complete: false,
    extracted_intent: intent,
    followup_questions: [],
    gatekeeper_reply:
      modelReply || modelFollowups.join(' ') || buildOutfitConfirmedGatekeeperReply(label),
  };
};

const handleOutfitSelection: RequestTypeHandler = ({ intent, modelReply, modelFollowups }) => {
  const label = outfitIdToLabel(intent.selected_outfit_id);
  console.log('[GATEKEEPER] outfit_selection — Gatekeeper 直接确认并追问，短路下游');
  return {
    is_complete: false,
    extracted_intent: intent,
    followup_questions: [],
    gatekeeper_reply:
      modelReply || modelFollowups.join(' ') || buildOutfitSelectionGatekeeperReply(label),
  };
};

const handleFeedbackRevision: RequestTypeHandler = (ctx) => {
  let intent = { ...ctx.intent, session_item_id: '' };
  const currentText = ctx.currentMessageText?.trim() ?? '';
  const llmSelected =
    intent.selected_outfit_id === 'outfit_1' || intent.selected_outfit_id === 'outfit_2'
      ? intent.selected_outfit_id
      : '';
  // 选套优先级：本轮原文 > 历史用户选套 > LLM（仅当 special_requests 能印证，防幻觉默认 outfit_1）
  const corroboratedLlmSelected =
    llmSelected && extractOutfitIdFromText(intent.special_requests) === llmSelected
      ? llmSelected
      : '';
  const resolvedOutfitId =
    extractOutfitIdFromText(currentText) ||
    (ctx.history ? inferSelectedOutfitIdFromHistory(ctx.history) : '') ||
    corroboratedLlmSelected;

  if (resolvedOutfitId) {
    intent = { ...intent, selected_outfit_id: resolvedOutfitId };
    return { is_complete: true, extracted_intent: intent, followup_questions: [] };
  }

  intent = { ...intent, selected_outfit_id: '' };
  const multipleOutfits = ctx.history ? historyShowsMultipleOutfits(ctx.history) : false;
```

```
  if (multipleOutfits) {
    console.log('[GATEKEEPER] feedback_revision — 多套方案且未选套，追问用户');
    return {
      is_complete: false,
      extracted_intent: intent,
      followup_questions: [],
      gatekeeper_reply: ctx.modelReply || buildFeedbackRevisionOutfitClarifyReply(),
    };
  }

  intent = { ...intent, selected_outfit_id: 'outfit_1' };
  return { is_complete: true, extracted_intent: intent, followup_questions: [] };
};

const handleWardrobePairing: RequestTypeHandler = (ctx) => {
  let intent = ctx.intent;
  const { modelReply, modelFollowups, wardrobeResolver: resolver, currentMessageText } = ctx;
  const browseOnly = isWardrobeBrowseIntent(intent, currentMessageText);

  if (resolver?.status === 'color_mismatch' && !intent.anchor_wardrobe_id?.trim()) {
    return {
      is_complete: false,
      extracted_intent: intent,
      followup_questions: [],
      gatekeeper_reply: buildColorMismatchReply(resolver.queriedSummary, resolver.nearMiss),
      wardrobe_candidates: [resolver.nearMiss],
    };
  }

  if (resolver?.status === 'not_found' && !intent.anchor_wardrobe_id?.trim()) {
    return {
      is_complete: false,
      extracted_intent: intent,
      followup_questions: [],
      gatekeeper_reply: modelReply || WARDROBE_NOT_FOUND_REPLY,
    };
  }

  if (resolver?.status === 'ambiguous' && !intent.anchor_wardrobe_id?.trim()) {
    return {
      is_complete: false,
      extracted_intent: intent,
      followup_questions: browseOnly ? [] : modelFollowups,
      gatekeeper_reply:
        modelReply ||
        (browseOnly
          ? buildWardrobeBrowseReply(resolver.candidates.length)
          : buildWardrobeAmbiguousReply()),
      wardrobe_candidates: resolver.candidates,
    };
```

```
  }

  if (resolver?.status === 'resolved' && !intent.anchor_wardrobe_id?.trim()) {
    intent = { ...intent, anchor_wardrobe_id: resolver.itemId };
  }

  if (browseOnly && resolver?.status === 'resolved') {
    return {
      is_complete: false,
      extracted_intent: intent,
      followup_questions: [],
      gatekeeper_reply: buildWardrobeBrowseReply(1),
      wardrobe_candidates: [resolver.item],
    };
  }

  if (!isWardrobePairingAnchorReady(intent)) {
    if (!intent.anchor_item_summary.trim()) {
      return {
        is_complete: false,
        extracted_intent: intent,
        followup_questions:
          modelFollowups.length > 0 ? modelFollowups : [PURCHASE_PAIRING_ANCHOR_FOLLOWUP],
      };
    }
    return {
      is_complete: false,
      extracted_intent: intent,
      followup_questions: [],
      gatekeeper_reply: modelReply || buildWardrobeAmbiguousReply(),
      wardrobe_candidates:
        resolver?.status === 'ambiguous' ? resolver.candidates : undefined,
    };
  }

  if (!intent.occasion.trim()) {
    return blockForMissingOccasion({ ...ctx, intent });
  }

  return { is_complete: true, extracted_intent: intent, followup_questions: [] };
};

const handlePurchasePairing: RequestTypeHandler = ({ intent, modelFollowups }) => {
  if (!isPurchasePairingAnchorReady(intent)) {
    console.warn('[GATEKEEPER] purchase_pairing blocked: missing anchor fields');
    return {
      is_complete: false,
      extracted_intent: intent,
      followup_questions:
        modelFollowups.length > 0 ? modelFollowups : [PURCHASE_PAIRING_ANCHOR_FOLLOWUP],
```

```
    };
  }
  return { is_complete: true, extracted_intent: intent, followup_questions: [] };
};

const handleStyleAdvice: RequestTypeHandler = ({ intent }) => {
  console.log('[GATEKEEPER] style_advice — 放行，由 Stylist 以建议模式回答');
  return { is_complete: true, extracted_intent: intent, followup_questions: [] };
};

const handleWardrobeOutfit: RequestTypeHandler = (ctx) => {
  if (!ctx.intent.occasion.trim()) {
    return blockForMissingOccasion(ctx);
  }
  return { is_complete: true, extracted_intent: ctx.intent, followup_questions: [] };
};

// ─── Handler registry ──────────────────────────────────────────────

/**
 * request_type → 处理策略 的穷尽式注册表。
 * 新增 OutfitRequestType 时，TS 会在此处强制要求补充对应 handler。
 */
const REQUEST_TYPE_HANDLERS: Record<OutfitRequestType, RequestTypeHandler> = {
  clarify: handleClarify,
  outfit_confirmed: handleOutfitConfirmed,
  outfit_selection: handleOutfitSelection,
  feedback_revision: handleFeedbackRevision,
  wardrobe_pairing: handleWardrobePairing,
  purchase_pairing: handlePurchasePairing,
  style_advice: handleStyleAdvice,
  wardrobe_outfit: handleWardrobeOutfit,
};

/** 全部合法 request_type（从注册表派生，单一真源，避免各处清单漂移） */
export const ALL_REQUEST_TYPES = Object.keys(REQUEST_TYPE_HANDLERS) as OutfitRequestType[];

export function isValidRequestType(value: string): value is OutfitRequestType {
  return Object.prototype.hasOwnProperty.call(REQUEST_TYPE_HANDLERS, value);
}

/** 硬校验放行条件，防止 LLM 误标 is_complete 导致 pairing 静默降级 */
export function finalizeGatekeeperResult(
  input: FinalizeGatekeeperInput
): GatekeeperFinalizeResult {
  const intent = normalizeGatekeeperIntent(input.extracted_intent);
  const handler = REQUEST_TYPE_HANDLERS[intent.request_type] ?? handleWardrobeOutfit;
  return handler({
    intent,
    modelFollowups: input.followup_questions?.filter((q) => q.trim()) ?? [],
```

```
    modelReply: input.gatekeeper_reply?.trim(),
    modelIsComplete: input.modelIsComplete,
    suggestCityForWeather: input.suggestCityForWeather,
    fetchedWeather: input.fetchedWeather,
    wardrobeResolver: input.wardrobeResolver,
    currentMessageText: input.currentMessageText,
    history: input.history,
  });
}


// ===== FILE: server/agents/intent/index.ts =====
/**
 * Intent types, utilities, and gatekeeper finalize registry.
 *
 *   typeDefs.ts  — types, interfaces, constants
 *   utils.ts     — text/slot/climate/weather utilities + intent normalization
 *   finalize.ts  — gatekeeper handler registry + finalizeGatekeeperResult
 */
export * from './typeDefs';
export * from './utils';
export * from './finalize';


// ===== FILE: server/agents/intent/typeDefs.ts =====
import { Content } from '@google/genai';

// ─── Request Types ──────────────────────────────────────────────────────────

export type OutfitRequestType =
  | 'wardrobe_outfit'
  | 'wardrobe_pairing'
  | 'purchase_pairing'
  | 'feedback_revision'
  | 'outfit_selection'
  | 'outfit_confirmed'
  | 'clarify'
  | 'style_advice';

export type AnchorSlot = 'top' | 'bottom' | 'dress' | 'shoes' | 'outerwear' | 'accessory';

/** 本轮搭配适用的穿衣气候，供 RAG 季节过滤；由 Gatekeeper 推断，服务端仅做锚点矛盾纠错 */
export type DressingClimate = 'cold' | 'warm' | 'mild';

/**
 * 本轮 city 对用户的含义：home=常住所在地（可写入档案）；travel=旅游/目的地（仅本轮天气，不写档案）；空=未提及或不适用。
 */
export type CityRole = 'home' | 'travel';

// ─── Interfaces ─────────────────────────────────────────────────────────────
```

```
export interface AnchorItemImageData {
  data: string;
  mimeType: string;
}

export interface AnchorItemInfo {
  name: string;
  summary: string;
  slot: AnchorSlot;
  imageData?: AnchorItemImageData;
  /** 待购锚点 COS URL（优先于 base64，跨轮稳定） */
  imageUrl?: string;
}

export interface GatekeeperIntent {
  weather: string;
  city?: string;
  /**
   * city 的语义角色：home 常住所在地可落档；travel 旅游/出行目的地不落档；空表示无城市或不适用。
   */
  city_role?: CityRole | '';
  occasion: string;
  style_preference: string;
  special_requests: string;
  request_type: OutfitRequestType;
  anchor_item_summary: string;
  /** wardrobe_pairing 时由 Gatekeeper LLM 生成的精简检索词，如「白色连衣裙」 */
  wardrobe_search_query?: string;
  anchor_slot: AnchorSlot | '';
  anchor_item_image_data?: AnchorItemImageData;
  /** 待购锚点图 COS URL；purchase_pairing / 微调继承时写入 */
  anchor_image_url?: string;
  /**
   * purchase_pairing：本会话待购清单中的 id（si_1…）；
   * feedback_revision 必须为空（绑定方案上的锚点，不绑「最新上传」）
   */
  session_item_id?: string;
  /** outfit_selection 时用户选中的方案 id，如 outfit_1 */
  selected_outfit_id?: string;
  /** wardrobe_pairing 时已确认的衣橱单品 id */
  anchor_wardrobe_id?: string;
  /** 本轮搭配的穿衣气候：cold 秋冬保暖 / warm 春夏轻薄 / mild 过渡季或场合不明确 */
  dressing_climate: DressingClimate | '';
}

export interface WardrobeAnchorCandidate {
  id: string;
  imageUrl: string;
  subCategory: string;
```

```
  colors: string[];
  similarity?: number;
}

export type WardrobeResolverResult =
  | { status: 'resolved'; itemId: string; item: WardrobeAnchorCandidate }
  | { status: 'ambiguous'; candidates: WardrobeAnchorCandidate[] }
  | { status: 'not_found' }
  | {
      status: 'color_mismatch';
      queriedSummary: string;
      nearMiss: WardrobeAnchorCandidate;
    };

export interface WeatherEnrichmentContext {
  clientIp?: string;
  profileLocation?: string;
  contextText?: string;
}

// ─── Finalize I/O interfaces ──────────────────────────────────────────

export interface FinalizeGatekeeperInput {
  extracted_intent: GatekeeperIntent;
  followup_questions?: string[];
  gatekeeper_reply?: string;
  suggestCityForWeather?: boolean;
  /** 服务端本轮按 weather_lookup 查到的天气；区别于 intent.weather 中用户自述的天气 */
  fetchedWeather?: string;
  wardrobeResolver?: WardrobeResolverResult;
  currentMessageText?: string;
  history?: Content[];
  /** Gatekeeper LLM 本轮自判的放行结论，仅 style_advice 等语义类型参考 */
  modelIsComplete?: boolean;
}

export interface GatekeeperFinalizeResult {
  is_complete: boolean;
  extracted_intent: GatekeeperIntent;
  followup_questions: string[];
  gatekeeper_reply?: string;
  wardrobe_candidates?: WardrobeAnchorCandidate[];
}

// ─── Defaults ─────────────────────────────────────────────────────────

export const DEFAULT_GATEKEEPER_INTENT: GatekeeperIntent = {
  weather: '',
  city: '',
  city_role: '',
```

```
  occasion: '',
  style_preference: '日常休闲',
  special_requests: '',
  request_type: 'wardrobe_outfit',
  anchor_item_summary: '',
  anchor_slot: '',
  session_item_id: '',
  anchor_image_url: '',
  dressing_climate: '',
};


// ===== FILE: server/agents/intent/utils.ts =====
import { Content, Part } from '@google/genai';
import { enrichWeather } from '@/server/services/amapWeatherService';
import {
  AnchorItemImageData,
  AnchorItemInfo,
  AnchorSlot,
  CityRole,
  DressingClimate,
  GatekeeperIntent,
  DEFAULT_GATEKEEPER_INTENT,
  WeatherEnrichmentContext,
} from './typeDefs';

/**
 * ─── LLM 输出纠偏规则清单（务必先读完再新增规则）──────────────────────
 *
 * 本文件里对 Gatekeeper LLM 输出做后处理的函数分两类：
 *
 * 【A类·空值兜底】只在 LLM 漏填某字段时补默认值，不覆盖 LLM 已做的分类判断。
 * 风险低，可以按需增加：
 *   - inferOccasionFromText
 *   - buildWardrobePairingSpecialRequest / buildPurchasePairingSpecialRequest
 *   - ACCESSORY_ANCHOR_PATTERN 匹配（enrichIntentFromContext 内）
 *   - extractConfirmedWardrobeId（解析系统自己生成的确认字符串，非猜测）
 * 天气城市：只认 Gatekeeper intent.city / weather_lookup.city 与 profileLocation，禁止从用户全文正则猜城。
 * 档案落盘：仅当 Gatekeeper city_role=home 时写入 location（travel 不覆盖）。
 *
 * 【B类·矛盾纠正/分类裁决】覆盖 LLM 已给出的显式判断，用于修正已知的
 * 系统性误判。这类规则有风险（可能误伤未覆盖到的正常场景），当前清单：
 *   1. correctAnchorSlot                — anchor_slot 与文本描述矛盾时纠正
 *   2. correctDressingClimate           — dressing_climate 与锚点描述矛盾时纠正
 *   3. coerceWardrobeBrowseIntent       — clarify → wardrobe_pairing（衣橱浏览类误判）
 *   4. adjudicateNewTaskVsRevisionIntent — feedback_revision → wardrobe_outfit（新场景/整轮重做误判为微调）
 *   5. resolveWardrobeAnchorIdFromHistory — 口头确认/幻觉 id 时用上轮展示的候选纠偏
 *
 * 新增 B 类规则前请先确认：
 *   - 能否通过调整 Gatekeeper prompt/schema 描述解决？优先改 prompt。
```

```
 *   - 触发条件是否足够窄、可用正则/关键词稳定判断？不要为单一场景写规则，
 *     应抽象成通用信号（如"是否有明确修改动作词"），否则会陷入无限打补丁。
 *   - 是否补充了对应单元测试？B 类规则必须有测试覆盖。
 *   - B 类规则数量建议控制在个位数；持续增长应考虑用统一的分类裁决器替代零散规则。
 *
 * 调用顺序（server/agents/gatekeeper/agent.ts）：
 *   rawIntent → coerceWardrobeBrowseIntent → adjudicateNewTaskVsRevisionIntent → enrichIntentFromContext
 * 两者作用的 request_type 互斥（clarify vs feedback_revision），暂无顺序冲突；
 * 新增规则时需重新评估是否存在交叉影响。
 * ──────────────────────────────────────────────────────────────────────
 */

// ─── Regex patterns ──────────────────────────────────────────────────────

export const ACCESSORY_ANCHOR_PATTERN =
  /耳环|耳钉|耳坠|项链|颈链|手链|手镯|戒指|吊坠|胸针|发夹|发饰|手表|腰带|皮带|围巾|丝巾|帽子|贝雷帽|棒球帽|手提包|单肩包|斜挎包|墨镜|眼镜|配饰|choker|earring|necklace|bracele...

const CASUAL_OCCASION_PATTERN = /平时|日常|百搭|通勤|上班|都可以穿|随便穿/i;

const OUTFIT_2_IN_TEXT =
  /第二套|二套|^二$|方案二|方案\s*2|outfit[_\s-]?2|第\s*2\s*套|(?<!\d)2\s*套|^2[.、)]?$/i;
const OUTFIT_1_IN_TEXT =
  /第一套|^一$|方案一|方案\s*1|outfit[_\s-]?1|第\s*1\s*套|(?<!\d)1\s*套|^1[.、)]?$/i;
const MULTIPLE_OUTFITS_IN_TEXT =
  /方案二|第二套|二套|outfit_2|第\s*2\s*套|两套方案|两套穿搭|准备了\s*2\s*套|两套/;

const ANCHOR_SLOT_SET = new Set<string>(['top', 'bottom', 'dress', 'shoes', 'outerwear', 'accessory']);
const DRESSING_CLIMATE_SET = new Set<string>(['cold', 'warm', 'mild']);

const REVISION_TARGET_PATTERN =
  /第一套|第二套|二套|^[一二]$|这套|那套|上一套|刚才那套|上面那套|方案一|方案二|outfit_[12]|第\s*[12]\s*套|(?<!\d)[12]\s*套|^[12][.、)]?$/i;
const REVISION_ACTION_PATTERN =
  /换成|改成|换一下|改一下|去掉|不要|保留|替换|调整|微调|加一件|加上|去除|换掉|太.{0,6}了|更.{0,6}一点/;
const FULL_REGEN_PATTERN =
  /都不好|都不满意|都不喜欢|全都不|全部重(?:做|来|新)|换一批|重新生成|再来一套|再来几套|重新(?:搭配|推荐|出方案)|搭配得?太烂|搭配太烂|太丑|太差|不喜欢这套|这套不行/;
const NEW_OUTFIT_REQUEST_PATTERN =
  /穿啥|穿什么|怎么穿|怎么搭|搭配一套|配一套|来一套|出一套|穿搭方案|应该穿|适合穿/;
const OCCASION_KEYWORDS =
  '打球|踢球|运动|健身|跑步|爬山|徒步|约会|通勤|上班|面试|聚会|旅行|逛街|打篮球|踢足球|篮球|足球';
const NEW_OCCASION_PATTERN = new RegExp(`(?:去.{0,8})?(?:${OCCASION_KEYWORDS})`);

const ANCHOR_COLD_CLIMATE_HINT =
  /冬季|冬天|寒冷|下雪|保暖|winter|cold|snow|羽绒|厚外套|毛呢|呢大衣|羊绒|棉服/i;
const ANCHOR_WARM_CLIMATE_HINT = /夏季|夏天|防晒|轻薄|短袖|短裤|吊带|海滩|海边|beach|summer/i;
const ANCHOR_OUTERWEAR_HINT = /外套|大衣|风衣|夹克|coat|jacket|parka|blazer|cardigan/i;

const WARDROBE_ID_CONFIRM_PATTERN =
  /(?:确认选择这件单品|就是这件|选这件|id=)\s*[(([]?([a-z0-9]{20,})/i;

/** History marker written by formatHistoryAsync for wardrobe candidate pickers. */
```

```
const WARDROBE_CANDIDATES_HISTORY_PATTERN = /\[wardrobe_candidates:id=([a-z0-9,]+)\]/gi;

/** User verbally accepts the (usually single) presented candidate without clicking id=. */
const VERBAL_ANCHOR_ACCEPT_PATTERN =
  /这个也行|那件也行|这件也行|也可以|就这件|就这个|就是它|用这件|穿这件|帮我(?:搭|配)|按这个|选这个|确认这|对这件|好吧这/;

const VERBAL_ANCHOR_REJECT_PATTERN =
  /不是这|不要这|不对|错了|换一|重新找|不是我要|我要别的/;

// ─── Text / History extraction ──────────────────────────────────────

export function extractTextFromParts(parts: Part[]): string {
  return parts
    .filter((part): part is { text: string } => 'text' in part && typeof part.text === 'string')
    .map((part) => part.text.trim())
    .filter(Boolean)
    .join('\n');
}

export function extractUserTextFromHistory(history: Content[]): string {
  return history
    .filter((msg) => msg.role === 'user')
    .flatMap((msg) => extractTextFromParts(msg.parts ?? []))
    .join('\n');
}

export function extractAssistantTextFromHistory(history: Content[]): string {
  return history
    .filter((msg) => msg.role === 'model')
    .flatMap((msg) => extractTextFromParts(msg.parts ?? []))
    .join('\n');
}

function isImagePart(part: Part): boolean {
  return 'inlineData' in part && Boolean(part.inlineData);
}

/** 收集历史中用户上传过的图片 + 当前输入的图片，用于跨轮次找回锚点图（如先发图后补场合）。 */
function collectImageParts(history: Content[], currentInput: Part[]): Part[] {
  const historyImages = history
    .filter((msg) => msg.role === 'user')
    .flatMap((msg) => (msg.parts ?? []).filter(isImagePart));
  return [...historyImages, ...currentInput.filter(isImagePart)];
}

export function collectLatestImageData(
  history: Content[],
  currentInput: Part[]
): AnchorItemImageData | undefined {
  const images = collectImageParts(history, currentInput);
```

```
  for (let i = images.length - 1; i >= 0; i--) {
    const inline = 'inlineData' in images[i] ? images[i].inlineData : undefined;
    if (inline?.data && inline?.mimeType) {
      return { data: inline.data, mimeType: inline.mimeType };
    }
  }
  return undefined;
}

// ——— Outfit selection helpers ———————————————————————————————

export function outfitIdToLabel(outfitId?: string): string {
  return outfitId === 'outfit_2' ? '第二套' : '第一套';
}

/** 从单条用户消息中提取明确选套信号 */
export function extractOutfitIdFromText(text: string): 'outfit_1' | 'outfit_2' | '' {
  const normalized = text.trim();
  if (!normalized) return '';
  if (OUTFIT_2_IN_TEXT.test(normalized)) return 'outfit_2';
  if (OUTFIT_1_IN_TEXT.test(normalized)) return 'outfit_1';
  return '';
}

export function userMentionedOutfitIdInText(text: string): boolean {
  return extractOutfitIdFromText(text) !== '';
}

/** 历史助手回复中是否出现过多套方案 */
export function historyShowsMultipleOutfits(history: Content[]): boolean {
  return MULTIPLE_OUTFITS_IN_TEXT.test(extractAssistantTextFromHistory(history));
}

/** 从用户历史消息（新→旧）推断已选方案 */
export function inferSelectedOutfitIdFromHistory(history: Content[]): 'outfit_1' | 'outfit_2' | '' {
  const userMessages = history
    .filter((msg) => msg.role === 'user')
    .map((msg) => extractTextFromParts(msg.parts ?? []))
    .filter(Boolean);

  for (let i = userMessages.length - 1; i >= 0; i--) {
    const id = extractOutfitIdFromText(userMessages[i]);
    if (id) return id;
  }
  return '';
}

// ——— Occasion / city / weather ——————————————————————————————

export function inferOccasionFromText(text: string): string {
```

```
  if (/上班|通勤|开会|办公室|工作/.test(text)) return '上班通勤';
  if (/徒步|爬山|户外/.test(text)) return '户外徒步';
  if (/约会/.test(text)) return '约会';
  if (/运动|健身|跑步|球/.test(text)) return '运动';
  if (/聚会|派对|晚宴|婚礼/.test(text)) return '聚会';
  if (/逛街|购物|商场/.test(text)) return '逛街';
  if (CASUAL_OCCASION_PATTERN.test(text)) return '日常百搭';
  return '';
}

/** 查天气用城市：仅 intent.city → 档案城市；空则交给 IP。禁止扫用户全文猜城。 */
export function resolveWeatherCity(
  intent: Pick<GatekeeperIntent, 'city'>,
  ctx: Pick<WeatherEnrichmentContext, 'profileLocation'>
): string {
  return intent.city?.trim() || ctx.profileLocation?.trim() || '';
}

export async function enrichIntentWeather(
  intent: GatekeeperIntent,
  ctx: WeatherEnrichmentContext
): Promise<GatekeeperIntent> {
  if (intent.weather.trim()) return intent;

  const city = resolveWeatherCity(intent, ctx);
  console.log(`[WEATHER] Fetching weather — city="${city || '(ip fallback)'}" ip="${ctx.clientIp || '-
'}"`);
  const weather = await enrichWeather({
    city: city || undefined,
    ip: ctx.clientIp,
  });
  if (weather) console.log(`[WEATHER] Result: ${weather}`);
  else console.log('[WEATHER] No weather data returned');

  if (!weather) return intent;

  return {
    ...intent,
    city: city || intent.city,
    weather,
  };
}

// ─── Anchor slot ──────────────────────────────────────────────────────────

export function parseAnchorSlot(value?: string): AnchorSlot | '' {
  const slot = value?.trim().toLowerCase();
  if (slot && ANCHOR_SLOT_SET.has(slot)) return slot as AnchorSlot;
  return '';
}
```

```
function inferAnchorSlotFromText(text: string): AnchorSlot | '' {
  const normalized = text.trim();
  if (!normalized) return '';
  if (ACCESSORY_ANCHOR_PATTERN.test(normalized)) return 'accessory';
  if (
    /外套|夹克|风衣|大衣|毛呢|呢大衣|羊绒|羽绒|棉服|jacket|coat|cardigan|windbreaker|blazer|parka/i.test(normaliz
ed)
  ) {
    return 'outerwear';
  }
  if (/连衣裙|连身裙|one.?piece|dress/i.test(normalized)) return 'dress';
  if (/半裙|短裙|A字裙|skirt/i.test(normalized)) return 'bottom';
  if (/裙(?!链|带)/i.test(normalized)) return 'dress';
  if (/裤|jeans|trousers|pants|shorts/i.test(normalized)) return 'bottom';
  if (/鞋|靴|sneaker|loafer|heel|boot/i.test(normalized)) return 'shoes';
  if (/衬衫|上衣|T恤|针织|背心|blouse|top|tee/i.test(normalized)) return 'top';
  return '';
}

/** Infer anchor slot from item description; summary takes priority over conversation context. */
export function inferAnchorSlotFromSummary(summary: string, contextText = ''): AnchorSlot | '' {
  const fromSummary = inferAnchorSlotFromText(summary);
  if (fromSummary) return fromSummary;
  return inferAnchorSlotFromText(contextText);
}

export function correctAnchorSlot(intent: GatekeeperIntent, contextText = ''): GatekeeperIntent {
  if (!isAnchorPairingIntent(intent)) return intent;

  const inferred = inferAnchorSlotFromSummary(intent.anchor_item_summary, contextText);
  if (inferred && intent.anchor_slot !== inferred) {
    console.warn(
      `[INTENT] Correcting anchor_slot: ${intent.anchor_slot || '(empty)'} → ${inferred} (${intent.anchor_
item_summar...
    );
    intent.anchor_slot = inferred;
  }
  return intent;
}

// ─── Dressing climate ──────────────────────────────────────────────────────

export function parseDressingClimate(value?: string): DressingClimate | '' {
  const normalized = value?.trim().toLowerCase();
  if (normalized && DRESSING_CLIMATE_SET.has(normalized)) {
    return normalized as DressingClimate;
  }
  return '';
}

export function parseCityRole(value?: string): CityRole | '' {
  const normalized = value?.trim().toLowerCase();
```

```
  }
}


// ===== FILE: app/api/wardrobe/[itemId]/route.ts =====
import { NextResponse } from 'next/server';

import { deleteWardrobeItems, getWardrobeItemDetails } from '@/server/services/wardrobeService';

export async function GET(
  req: Request,
  { params }: { params: Promise<{ itemId: string }> }
) {
  const { itemId } = await params;
  // 我们从请求头中获取客户端 ID，这是 API 的标准做法
  const clientId = req.headers.get('x-client-id');

  if (!clientId) {
    return NextResponse.json({ error: 'Client ID is required' }, { status: 400 });
  }

  if (!itemId) {
    return NextResponse.json({ error: 'Item ID is required' }, { status: 400 });
  }

  try {
    const item = await getWardrobeItemDetails(itemId);

    if (!item) {
      return NextResponse.json({ error: 'Item not found' }, { status: 404 });
    }

    // --- 关键安全检查 ---
    // 为防止信息泄露，如果物品不属于当前用户，我们同样返回 404
    if (item.clientProfileId !== clientId) {
      return NextResponse.json({ error: 'Item not found' }, { status: 404 });
    }

    // 准备返回给客户端的数据
    const responseData = {
      id: item.id,
      name: item.subCategory,
      imageUrl: item.imageUrl,
    };

    // 返回成功响应，并设置缓存头
    return NextResponse.json(responseData, {
      status: 200,
      headers: {
        // 在浏览器和 CDN 上缓存 1 小时
```

```
        'Cache-Control': 'public, max-age=3600, s-maxage=3600',
      },
    });

  } catch (error) {
    console.error(`[API-ERROR] Failed to fetch wardrobe item ${itemId}:`, error);
    return NextResponse.json({ error: 'Internal Server Error' }, { status: 500 });
  }
}

export async function DELETE(
  req: Request,
  { params }: { params: Promise<{ itemId: string }> }
) {
  const { itemId } = await params;
  const clientId = req.headers.get('x-client-id');

  if (!clientId) {
    return NextResponse.json({ error: 'Client ID is required' }, { status: 400 });
  }

  if (!itemId) {
    return NextResponse.json({ error: 'Item ID is required' }, { status: 400 });
  }

  try {
    const deletedCount = await deleteWardrobeItems([itemId], clientId);
    if (deletedCount === 0) {
      return NextResponse.json({ error: 'Item not found' }, { status: 404 });
    }

    return NextResponse.json({ deletedCount }, { status: 200 });
  } catch (error) {
    console.error(`[API-ERROR] Failed to delete wardrobe item ${itemId}:`, error);
    return NextResponse.json({ error: 'Internal Server Error' }, { status: 500 });
  }
}

// ===== FILE: app/api/wardrobe/route.ts =====
import { NextResponse } from 'next/server';
import prismadb from 'server/db';
import { llmGenerate } from '@/server/services/llm/client';
import { extractJsonText } from '@/server/services/llm/convert';
import { AGENT_MODELS } from '@/server/config/models';
import { urlToGenerativePart } from '@/server/utils/image';
import {
  clothingItemHasTextEmbedding,
  clothingItemHasVisualEmbedding,
  deleteWardrobeItems,
  findWardrobeItemByImageUrl,
```

```
  persistTextEmbedding,
  persistVisualEmbedding,
} from '@/server/services/wardrobeService';
import { is429Error, withRetryOn429 } from '@/server/utils/retryOn429';
import { toUserFacingUploadError } from '@/lib/upload-errors';
import {
  normalizeWardrobeAnalysis,
  WARDROBE_ANALYSIS_JSON_SCHEMA,
  WARDROBE_ANALYSIS_PROMPT,
  type WardrobeAnalysisResult,
} from '@/server/utils/wardrobeAnalysis';

export async function GET(req: Request) {
  try {
    const clientId = req.headers.get('X-Client-ID');
    if (!clientId) {
      return NextResponse.json({ error: 'X-Client-ID header is required' }, { status: 400 });
    }

    console.log(`[API /api/wardrobe] GET request for clientId: ${clientId}`);

    const clothingItems = await prismadb.clothingItem.findMany({
      where: { clientProfileId: clientId },
      orderBy: { createdAt: 'desc' },
    });

    return NextResponse.json(clothingItems, { status: 200 });
  } catch (error) {
    console.error('Error in GET /api/wardrobe:', error);
    return NextResponse.json({ error: 'Internal Server Error' }, { status: 500 });
  }
}

type StepState = 'pending' | 'done' | 'skipped' | 'failed';

export interface WardrobeStepStatus {
  analysis: StepState;
  textEmbedding: StepState;
  visualEmbedding: StepState;
}

function failStep(
  timer: ReturnType<typeof createStepTimer>,
  steps: WardrobeStepStatus,
  step: keyof WardrobeStepStatus,
  message: string,
  status: number,
  extra?: Record<string, unknown>
) {
  const finalSteps: WardrobeStepStatus = { ...steps, [step]: 'failed' };
```

```
  timer.log('error', { steps: finalSteps, ...extra });
  return NextResponse.json({ error: message, steps: finalSteps, ...extra }, { status });
}

function createStepTimer() {
  const startedAt = Date.now();
  let lastMark = startedAt;
  const ms: Record<string, number> = {};

  return {
    mark(step: string) {
      const now = Date.now();
      ms[step] = now - lastMark;
      lastMark = now;
    },
    log(outcome: 'success' | 'error', extra?: Record<string, unknown>) {
      ms.total = Date.now() - startedAt;
      console.log('[API /api/wardrobe] timing', JSON.stringify({ outcome, ...extra, ms }));
    },
  };
}

function isRateLimitResponse(error: unknown, steps: WardrobeStepStatus, item: { id: string }) {
  if (!is429Error(error)) return null;
  return NextResponse.json(
    { error: 'AI 服务请求过于频繁，请稍后再试', code: 'RATE_LIMIT', steps, item },
    { status: 429 }
  );
}

export async function POST(req: Request) {
  const timer = createStepTimer();
  try {
    const clientId = req.headers.get('X-Client-ID');
    if (!clientId) {
      return NextResponse.json({ error: 'X-Client-ID header is required' }, { status: 400 });
    }

    const { imageUrl } = await req.json();
    if (!imageUrl || typeof imageUrl !== 'string') {
      return NextResponse.json({ error: 'imageUrl is required in the request body' }, { status: 400 });
    }

    console.log(`[API /api/wardrobe] Received request for clientId: ${clientId}, imageUrl: ${imageUrl}`);

    let item = await findWardrobeItemByImageUrl(clientId, imageUrl);
    const steps: WardrobeStepStatus = {
      analysis: item ? 'skipped' : 'pending',
      textEmbedding: 'pending',
      visualEmbedding: 'pending',
```

```
    };
    timer.mark('lookup_existing');

    if (!item) {
      const imagePart = await urlToGenerativePart(imageUrl);
      timer.mark('fetch_image');

      console.log('[API /api/wardrobe] Calling LLM for analysis...', {
        model: AGENT_MODELS.wardrobeAnalysis,
      });
      const result = await withRetryOn429(
        () =>
          llmGenerate({
            model: AGENT_MODELS.wardrobeAnalysis,
            contents: [{ role: 'user', parts: [imagePart, { text: WARDROBE_ANALYSIS_PROMPT }] }],
            jsonSchema: WARDROBE_ANALYSIS_JSON_SCHEMA,
          }),
        { label: 'Wardrobe analysis', maxRetries: 4 }
      );
      timer.mark('llm_analysis');

      const responseText = result.text;
      console.log(`[API /api/wardrobe] LLM response received: ${responseText}`);

      if (!responseText) {
        return failStep(timer, steps, 'analysis', 'AI 分析没有返回内容，请重试', 502);
      }

      let parsed: WardrobeAnalysisResult;
      try {
        parsed = JSON.parse(extractJsonText(responseText));
      } catch {
        console.error('[API /api/wardrobe] Failed to parse JSON from AI response:', responseText);
        return failStep(timer, steps, 'analysis', 'AI 分析结果异常，请重试', 502);
      }

      const analysis = normalizeWardrobeAnalysis(parsed);
      if (!analysis) {
        console.error('[API /api/wardrobe] Invalid analysis payload:', parsed);
        return failStep(timer, steps, 'analysis', 'AI 分析结果异常，请重试', 502);
      }
      timer.mark('parse_and_validate');

      console.log('[API /api/wardrobe] Storing new clothing item to database...');
      item = await prismadb.clothingItem.create({
        data: {
          clientProfileId: clientId,
          imageUrl,
          mainCategory: analysis.mainCategory,
          subCategory: analysis.subCategory,
```

```
        season: analysis.season,
        material: analysis.material,
        colors: analysis.colors,
        tags: analysis.tags,
        description: analysis.description,
        searchDescription: analysis.searchDescription,
        occasions: analysis.occasions,
        formality: analysis.formality,
        silhouette: analysis.silhouette,
      },
    });
    steps.analysis = 'done';
    timer.mark('db_write');
  }

  if (await clothingItemHasTextEmbedding(item.id)) {
    steps.textEmbedding = 'skipped';
  } else {
    try {
      await persistTextEmbedding(item.id);
      steps.textEmbedding = 'done';
      timer.mark('text_embedding');
    } catch (error) {
      console.error('[API /api/wardrobe] Text embedding step failed', { itemId: item.id, error });
      const rateLimit = isRateLimitResponse(error, steps, item);
      if (rateLimit) {
        timer.log('error', { itemId: item.id, steps: { ...steps, textEmbedding: 'failed' } });
        return rateLimit;
      }
      return failStep(
        timer,
        steps,
        'textEmbedding',
        '文本检索向量生成失败，点击重试即可（分析结果已保存）',
        502,
        { item }
      );
    }
  }

  if (await clothingItemHasVisualEmbedding(item.id)) {
    steps.visualEmbedding = 'skipped';
  } else {
    try {
      await persistVisualEmbedding(item.id);
      steps.visualEmbedding = 'done';
      timer.mark('visual_embedding');
    } catch (error) {
      console.error('[API /api/wardrobe] Visual embedding step failed', { itemId: item.id, error });
      const rateLimit = isRateLimitResponse(error, steps, item);
```

```
        if (rateLimit) {
          timer.log('error', { itemId: item.id, steps: { ...steps, visualEmbedding: 'failed' } });
          return rateLimit;
        }
        return failStep(
          timer,
          steps,
          'visualEmbedding',
          '图片向量生成失败，点击重试即可（分析与文本向量已保存）',
          502,
          { item }
        );
      }
    }

    timer.log('success', {
      model: AGENT_MODELS.wardrobeAnalysis,
      itemId: item.id,
      subCategory: item.subCategory,
      steps,
    });

    return NextResponse.json({ ...item, steps }, { status: 201 });
  } catch (error) {
    timer.log('error', { model: AGENT_MODELS.wardrobeAnalysis });
    console.error('Error in POST /api/wardrobe:', error);
    if (is429Error(error)) {
      return NextResponse.json(
        { error: 'AI 服务请求过于频繁，请稍后再试', code: 'RATE_LIMIT' },
        { status: 429 }
      );
    }
    if (error instanceof Error && error.message.includes('GoogleGenerativeAI')) {
      return NextResponse.json({ error: 'AI 服务暂时不可用，请稍后重试' }, { status: 502 });
    }
    if (error instanceof Error) {
      return NextResponse.json({ error: toUserFacingUploadError(error.message) }, { status: 500 });
    }
    return NextResponse.json({ error: '处理失败，请重试' }, { status: 500 });
  }
}

export async function DELETE(req: Request) {
  try {
    const clientId = req.headers.get('X-Client-ID');
    if (!clientId) {
      return NextResponse.json({ error: 'X-Client-ID header is required' }, { status: 400 });
    }

    const body = (await req.json()) as { ids?: string[] };
```

```
    const ids = Array.isArray(body.ids) ? body.ids : [];

    if (ids.length === 0) {
      return NextResponse.json({ error: 'ids array is required' }, { status: 400 });
    }

    const deletedCount = await deleteWardrobeItems(ids, clientId);
    return NextResponse.json({ deletedCount }, { status: 200 });
  } catch (error) {
    console.error('Error in DELETE /api/wardrobe:', error);
    return NextResponse.json({ error: 'Internal Server Error' }, { status: 500 });
  }
}


// ===== FILE: app/api/auth/change-password/route.ts =====
import { NextRequest, NextResponse } from 'next/server';
import { getSessionUser } from '@/server/auth/currentUser';
// [REDACTED: sensitive literal removed for soft-copyright deposit]
import { createSessionToken } from '@/server/auth/session';
import { setSessionCookie } from '@/server/auth/sessionCookie';
import prismadb from '@/server/db';
import { logAuthEvent } from '@/server/logging/authAudit';

// [REDACTED: sensitive literal removed for soft-copyright deposit]
// [REDACTED: sensitive literal removed for soft-copyright deposit]
// [REDACTED: sensitive literal removed for soft-copyright deposit]
}

export async function POST(req: NextRequest) {
  const user = await getSessionUser(req);
  if (!user) {
    return NextResponse.json({ error: '请先登录', code: 'UNAUTHENTICATED' }, { status: 401 });
  }

// [REDACTED: sensitive literal removed for soft-copyright deposit]
// [REDACTED: sensitive literal removed for soft-copyright deposit]
// [REDACTED: sensitive literal removed for soft-copyright deposit]

// [REDACTED: sensitive literal removed for soft-copyright deposit]
  if (strengthError) {
    return NextResponse.json({ error: strengthError, code: 'VALIDATION_ERROR' }, { status: 400 });
  }

  try {
// [REDACTED: sensitive literal removed for soft-copyright deposit]
    if (!oldOk) {
      await logAuthEvent({
// [REDACTED: sensitive literal removed for soft-copyright deposit]
        outcome: 'failure',
```

```
        username: user.username,
        userId: user.id,
// [REDACTED: sensitive literal removed for soft-copyright deposit]
      });
      return NextResponse.json(
// [REDACTED: sensitive literal removed for soft-copyright deposit]
        { status: 403 }
      );
    }

    // bump sessionVersion：其他设备的 session 全部失效，当前会话续签
    const nextVersion = user.sessionVersion + 1;
    await prismadb.user.update({
      where: { id: user.id },
// [REDACTED: sensitive literal removed for soft-copyright deposit]
    });

    const token = await createSessionToken(user.id, nextVersion);
    const response = NextResponse.json({ ok: true });
    setSessionCookie(response, token);
    await logAuthEvent({
// [REDACTED: sensitive literal removed for soft-copyright deposit]
      outcome: 'success',
      username: user.username,
      userId: user.id,
    });
    return response;
  } catch (error) {
// [REDACTED: sensitive literal removed for soft-copyright deposit]
    return NextResponse.json({ error: 'Internal Server Error' }, { status: 500 });
  }
}


// ===== FILE: app/api/auth/login/route.ts =====
import { NextRequest, NextResponse } from 'next/server';
import { normalizeUsername } from '@/server/auth/credentials';
import { getLockStatus, registerFailure, resetFailures } from '@/server/auth/lockout';
// [REDACTED: sensitive literal removed for soft-copyright deposit]
import { createSessionToken } from '@/server/auth/session';
import { setSessionCookie } from '@/server/auth/sessionCookie';
import prismadb from '@/server/db';
import { logAuthEvent } from '@/server/logging/authAudit';

interface LoginBody {
  username?: unknown;
// [REDACTED: sensitive literal removed for soft-copyright deposit]
}

function lockedResponse(retryAfterSec: number) {
```

```
  return NextResponse.json(
    {
      error: `失败次数过多，账号已锁定，请 ${Math.ceil(retryAfterSec / 60)} 分钟后再试`,
      code: 'ACCOUNT_LOCKED',
      retryAfterSec,
    },
    { status: 423 }
  );
}

export async function POST(req: NextRequest) {
  const body = (await req.json().catch(() => null)) as LoginBody | null;
  const username = normalizeUsername(typeof body?.username === 'string' ? body.username : '');
// [REDACTED: sensitive literal removed for soft-copyright deposit]
// [REDACTED: sensitive literal removed for soft-copyright deposit]
    return NextResponse.json(
      { error: '请输入用户名和密码', code: 'VALIDATION_ERROR' },
      { status: 400 }
    );
  }

  try {
    const user = await prismadb.user.findUnique({ where: { username } });
    if (!user) {
      await logAuthEvent({ action: 'login', outcome: 'failure', username, reason: 'credentials_invalid'
});
      return NextResponse.json(
        { error: '用户名或密码不正确', code: 'CREDENTIALS_INVALID' },
        { status: 401 }
      );
    }

    const lock = getLockStatus(user);
    if (lock.locked) {
      await logAuthEvent({ action: 'login', outcome: 'failure', username, reason: 'account_locked' });
      return lockedResponse(lock.retryAfterSec);
    }

// [REDACTED: sensitive literal removed for soft-copyright deposit]
    if (!ok) {
      const next = registerFailure(user);
      await prismadb.user.update({ where: { id: user.id }, data: next });
      await logAuthEvent({ action: 'login', outcome: 'failure', username, reason: 'credentials_invalid'
});
      const nowLocked = getLockStatus(next);
      if (nowLocked.locked) return lockedResponse(nowLocked.retryAfterSec);
      return NextResponse.json(
        { error: '用户名或密码不正确', code: 'CREDENTIALS_INVALID' },
        { status: 401 }
      );
    }
```

```
    await prismadb.user.update({ where: { id: user.id }, data: resetFailures() });
    const token = await createSessionToken(user.id, user.sessionVersion);
    const response = NextResponse.json({ user: { id: user.id, username: user.username } });
    setSessionCookie(response, token);
    await logAuthEvent({ action: 'login', outcome: 'success', username, userId: user.id });
    return response;
  } catch (error) {
    console.error('[AUTH_LOGIN]', error);
    return NextResponse.json({ error: 'Internal Server Error' }, { status: 500 });
  }
}


// ===== FILE: app/api/auth/logout/route.ts =====
import { NextRequest, NextResponse } from 'next/server';
import { getSessionUser } from '@/server/auth/currentUser';
import { clearSessionCookie } from '@/server/auth/sessionCookie';
import { logAuthEvent } from '@/server/logging/authAudit';

export async function POST(req: NextRequest) {
  const user = await getSessionUser(req);
  const response = NextResponse.json({ ok: true });
  clearSessionCookie(response);
  await logAuthEvent({
    action: 'logout',
    outcome: 'success',
    username: user?.username,
    userId: user?.id,
  });
  return response;
}


// ===== FILE: app/api/auth/recovery-codes/route.ts =====
import { NextRequest, NextResponse } from 'next/server';
import { generateRecoveryCodes, normalizeRecoveryCode } from '@/server/auth/credentials';
import { getSessionUser } from '@/server/auth/currentUser';
// [REDACTED: sensitive literal removed for soft-copyright deposit]
import prismadb from '@/server/db';
import { logAuthEvent } from '@/server/logging/authAudit';

/** POST：登录态重新生成 8 个恢复码（旧码全部作废；明文仅本次返回） */
export async function POST(req: NextRequest) {
  const user = await getSessionUser(req);
  if (!user) {
    return NextResponse.json({ error: '请先登录', code: 'UNAUTHENTICATED' }, { status: 401 });
  }

  try {
    const codes = generateRecoveryCodes(8);
```

```
    const hashes: string[] = [];
    for (const code of codes) {
// [REDACTED: sensitive literal removed for soft-copyright deposit]
    }

    await prismadb.user.update({
      where: { id: user.id },
      data: { recoveryCodeHashes: hashes },
    });
    await logAuthEvent({
      action: 'generate_recovery_codes',
      outcome: 'success',
      username: user.username,
      userId: user.id,
    });
    return NextResponse.json({ codes });
  } catch (error) {
    console.error('[AUTH_RECOVERY_CODES]', error);
    return NextResponse.json({ error: 'Internal Server Error' }, { status: 500 });
  }
}


// ===== FILE: app/api/auth/register/route.ts =====
import { NextRequest, NextResponse } from 'next/server';
import { isAccessCodeValid } from '@/server/auth/access';
import { normalizeUsername, validateUsername } from '@/server/auth/credentials';
// [REDACTED: sensitive literal removed for soft-copyright deposit]
import { createSessionToken } from '@/server/auth/session';
import { setSessionCookie } from '@/server/auth/sessionCookie';
import prismadb from '@/server/db';
import { logAuthEvent } from '@/server/logging/authAudit';

interface RegisterBody {
  username?: unknown;
// [REDACTED: sensitive literal removed for soft-copyright deposit]
  inviteCode?: unknown;
}

export async function POST(req: NextRequest) {
  const body = (await req.json().catch(() => null)) as RegisterBody | null;
  const username = normalizeUsername(typeof body?.username === 'string' ? body.username : '');
// [REDACTED: sensitive literal removed for soft-copyright deposit]

  const usernameError = validateUsername(username);
  if (usernameError) {
    return NextResponse.json({ error: usernameError, code: 'VALIDATION_ERROR' }, { status: 400 });
  }
// [REDACTED: sensitive literal removed for soft-copyright deposit]
// [REDACTED: sensitive literal removed for soft-copyright deposit]
```

```
// [REDACTED: sensitive literal removed for soft-copyright deposit]
  }

  // 邀请码：REGISTRATION_MODE=open 时跳过；默认 invite，复用 ACCESS_CODE
  const inviteRequired = (process.env.REGISTRATION_MODE?.trim() || 'invite') !== 'open';
  const inviteCode = typeof body?.inviteCode === 'string' ? body.inviteCode : '';
  if (inviteRequired && !isAccessCodeValid(inviteCode)) {
    await logAuthEvent({ action: 'register', outcome: 'failure', username, reason: 'invite_invalid' });
    return NextResponse.json({ error: '邀请码不正确', code: 'INVITE_INVALID' }, { status: 403 });
  }

  try {
    const existing = await prismadb.user.findUnique({ where: { username } });
    if (existing) {
      await logAuthEvent({ action: 'register', outcome: 'failure', username, reason: 'username_taken' });
      return NextResponse.json({ error: '用户名已被占用', code: 'USERNAME_TAKEN' }, { status: 409 });
    }

// [REDACTED: sensitive literal removed for soft-copyright deposit]
    const user = await prismadb.user.create({
      data: {
        username,
// [REDACTED: sensitive literal removed for soft-copyright deposit]
        clientProfile: { create: {} },
      },
    });

    const token = await createSessionToken(user.id, user.sessionVersion);
    const response = NextResponse.json({ user: { id: user.id, username: user.username } });
    setSessionCookie(response, token);
    await logAuthEvent({ action: 'register', outcome: 'success', username, userId: user.id });
    return response;
  } catch (error) {
    console.error('[AUTH_REGISTER]', error);
    return NextResponse.json({ error: 'Internal Server Error' }, { status: 500 });
  }
}


// ===== FILE: app/api/auth/reset-password/route.ts =====
import { NextRequest, NextResponse } from 'next/server';
import {
  normalizeRecoveryCode,
  normalizeSecurityAnswer,
  normalizeUsername,
} from '@/server/auth/credentials';
import { getLockStatus, registerFailure, resetFailures } from '@/server/auth/lockout';
// [REDACTED: sensitive literal removed for soft-copyright deposit]
import { createSessionToken } from '@/server/auth/session';
import { setSessionCookie } from '@/server/auth/sessionCookie';
```

```
import prismadb from '@/server/db';
import { logAuthEvent } from '@/server/logging/authAudit';

// [REDACTED: sensitive literal removed for soft-copyright deposit]
  username?: unknown;
  method?: unknown;
  answers?: unknown;
  recoveryCode?: unknown;
// [REDACTED: sensitive literal removed for soft-copyright deposit]
}

interface StoredSecurityQuestion {
  question: string;
  answerHash: string;
}

function parseStoredQuestions(raw: unknown): StoredSecurityQuestion[] {
  if (!Array.isArray(raw)) return [];
  return raw.filter(
    (item): item is StoredSecurityQuestion =>
      typeof item === 'object' &&
      item !== null &&
      typeof (item as StoredSecurityQuestion).question === 'string' &&
      typeof (item as StoredSecurityQuestion).answerHash === 'string'
  );
}

function lockedResponse(retryAfterSec: number) {
  return NextResponse.json(
    {
      error: `失败次数过多，账号已锁定，请 ${Math.ceil(retryAfterSec / 60)} 分钟后再试`,
      code: 'ACCOUNT_LOCKED',
      retryAfterSec,
    },
    { status: 423 }
  );
}

export async function POST(req: NextRequest) {
// [REDACTED: sensitive literal removed for soft-copyright deposit]
  const username = normalizeUsername(typeof body?.username === 'string' ? body.username : '');
  const method = body?.method === 'recovery' ? 'recovery' : body?.method === 'questions' ? 'questions' : n
ull;
// [REDACTED: sensitive literal removed for soft-copyright deposit]

  if (!username || !method) {
    return NextResponse.json({ error: '请求参数不完整', code: 'VALIDATION_ERROR' }, { status: 400 });
  }
// [REDACTED: sensitive literal removed for soft-copyright deposit]
  if (strengthError) {
    return NextResponse.json({ error: strengthError, code: 'VALIDATION_ERROR' }, { status: 400 });
```

```
  }

  try {
    const user = await prismadb.user.findUnique({ where: { username } });
    if (!user) {
      return NextResponse.json(
        { error: '重置失败，请检查输入', code: 'RESET_FAILED' },
        { status: 403 }
      );
    }

    const lock = getLockStatus(user);
    if (lock.locked) {
// [REDACTED: sensitive literal removed for soft-copyright deposit]
      return lockedResponse(lock.retryAfterSec);
    }

    let remainingRecoveryHashes: string[] | null = null;

    if (method === 'questions') {
      const stored = parseStoredQuestions(user.securityQuestions);
      if (stored.length === 0) {
        return NextResponse.json(
          { error: '该账号未设置密保问题，请使用恢复码或联系管理员', code: 'RESET_UNAVAILABLE' },
          { status: 403 }
        );
      }
      const answers = Array.isArray(body?.answers) ? body.answers : [];
      if (answers.length !== stored.length) {
        return NextResponse.json(
          { error: '请回答全部密保问题', code: 'VALIDATION_ERROR' },
          { status: 400 }
        );
      }
      let allMatch = true;
      for (let i = 0; i < stored.length; i += 1) {
        const answer = typeof answers[i] === 'string' ? answers[i] : '';
// [REDACTED: sensitive literal removed for soft-copyright deposit]
        if (!ok) allMatch = false;
      }
      if (!allMatch) {
        const next = registerFailure(user);
        await prismadb.user.update({ where: { id: user.id }, data: next });
// [REDACTED: sensitive literal removed for soft-copyright deposit]
        const nowLocked = getLockStatus(next);
        if (nowLocked.locked) return lockedResponse(nowLocked.retryAfterSec);
        return NextResponse.json(
          { error: '密保答案不正确', code: 'ANSWERS_MISMATCH' },
          { status: 403 }
        );
```

```
      }
    } else {
      // recovery：任一未使用恢复码匹配即通过，命中后消费该码
      const code = normalizeRecoveryCode(
        typeof body?.recoveryCode === 'string' ? body.recoveryCode : ''
      );
      if (!code) {
        return NextResponse.json({ error: '请输入恢复码', code: 'VALIDATION_ERROR' }, { status: 400 });
      }
      let matchedIndex = -1;
      for (let i = 0; i < user.recoveryCodeHashes.length; i += 1) {
// [REDACTED: sensitive literal removed for soft-copyright deposit]
          matchedIndex = i;
          break;
        }
      }
      if (matchedIndex < 0) {
        const next = registerFailure(user);
        await prismadb.user.update({ where: { id: user.id }, data: next });
// [REDACTED: sensitive literal removed for soft-copyright deposit]
        const nowLocked = getLockStatus(next);
        if (nowLocked.locked) return lockedResponse(nowLocked.retryAfterSec);
        return NextResponse.json(
          { error: '恢复码不正确或已被使用', code: 'RECOVERY_INVALID' },
          { status: 403 }
        );
      }
      remainingRecoveryHashes = user.recoveryCodeHashes.filter((_, i) => i !== matchedIndex);
    }

    // 重置成功：新密码 + sessionVersion+1（旧 session 全失效）+ 清锁定，当前会话自动登录
    const nextVersion = user.sessionVersion + 1;
    await prismadb.user.update({
      where: { id: user.id },
      data: {
// [REDACTED: sensitive literal removed for soft-copyright deposit]
        sessionVersion: nextVersion,
        ...resetFailures(),
        ...(remainingRecoveryHashes ? { recoveryCodeHashes: remainingRecoveryHashes } : {}),
      },
    });

    const token = await createSessionToken(user.id, nextVersion);
    const response = NextResponse.json({ ok: true });
    setSessionCookie(response, token);
    await logAuthEvent({
// [REDACTED: sensitive literal removed for soft-copyright deposit]
      outcome: 'success',
      username,
      userId: user.id,
```

```
    });
    return response;
  } catch (error) {
// [REDACTED: sensitive literal removed for soft-copyright deposit]
    return NextResponse.json({ error: 'Internal Server Error' }, { status: 500 });
  }
}


// ===== FILE: app/api/auth/security-questions/route.ts =====
import type { Prisma } from '@prisma/client';
import { NextRequest, NextResponse } from 'next/server';
import {
  normalizeSecurityAnswer,
  normalizeUsername,
  validateSecurityQuestions,
  type SecurityQuestionInput,
} from '@/server/auth/credentials';
import { getSessionUser } from '@/server/auth/currentUser';
// [REDACTED: sensitive literal removed for soft-copyright deposit]
import prismadb from '@/server/db';
import { logAuthEvent } from '@/server/logging/authAudit';

interface StoredSecurityQuestion {
  question: string;
  answerHash: string;
}

function parseStoredQuestions(raw: unknown): StoredSecurityQuestion[] {
  if (!Array.isArray(raw)) return [];
  return raw.filter(
    (item): item is StoredSecurityQuestion =>
      typeof item === 'object' &&
      item !== null &&
      typeof (item as StoredSecurityQuestion).question === 'string' &&
      typeof (item as StoredSecurityQuestion).answerHash === 'string'
  );
}

/** GET ?username=x：找回密码时取问题文本；用户不存在统一返回空数组（避免用户名枚举） */
export async function GET(req: NextRequest) {
  const username = normalizeUsername(req.nextUrl.searchParams.get('username') ?? '');
  if (!username) {
    return NextResponse.json({ questions: [] });
  }

  try {
    const user = await prismadb.user.findUnique({ where: { username } });
    const questions = user ? parseStoredQuestions(user.securityQuestions) : [];
    return NextResponse.json({ questions: questions.map((q) => q.question) });
```

```
  } catch (error) {
    console.error('[AUTH_SECURITY_QUESTIONS_GET]', error);
    return NextResponse.json({ error: 'Internal Server Error' }, { status: 500 });
  }
}

interface SetQuestionsBody {
  questions?: unknown;
}

/** POST：登录态设置/覆盖密保问题（答案归一化后哈希存储） */
export async function POST(req: NextRequest) {
  const user = await getSessionUser(req);
  if (!user) {
    return NextResponse.json({ error: '请先登录', code: 'UNAUTHENTICATED' }, { status: 401 });
  }

  const body = (await req.json().catch(() => null)) as SetQuestionsBody | null;
  const inputs = Array.isArray(body?.questions) ? (body.questions as SecurityQuestionInput[]) : [];

  const validationError = validateSecurityQuestions(inputs);
  if (validationError) {
    return NextResponse.json({ error: validationError, code: 'VALIDATION_ERROR' }, { status: 400 });
  }

  try {
    const stored: StoredSecurityQuestion[] = [];
    for (const input of inputs) {
      stored.push({
        question: input.question.trim(),
// [REDACTED: sensitive literal removed for soft-copyright deposit]
      });
    }

    await prismadb.user.update({
      where: { id: user.id },
      data: { securityQuestions: stored as unknown as Prisma.InputJsonValue },
    });
    await logAuthEvent({
      action: 'set_security_questions',
      outcome: 'success',
      username: user.username,
      userId: user.id,
    });
    return NextResponse.json({ ok: true });
  } catch (error) {
    console.error('[AUTH_SECURITY_QUESTIONS_POST]', error);
    return NextResponse.json({ error: 'Internal Server Error' }, { status: 500 });
  }
}
```

```
// ===== FILE: app/api/auth/session/route.ts =====
import { NextRequest, NextResponse } from 'next/server';
import { getSessionUser } from '@/server/auth/currentUser';

/** 登录态探针：永不 401，前端据此决定是否跳登录页 */
export async function GET(req: NextRequest) {
  const user = await getSessionUser(req);
  if (!user) {
    return NextResponse.json({ authenticated: false });
  }

  const questions = Array.isArray(user.securityQuestions) ? user.securityQuestions : [];
  return NextResponse.json({
    authenticated: true,
    user: {
      id: user.id,
      username: user.username,
      hasSecurityQuestions: questions.length > 0,
    },
  });
}


// ===== FILE: app/api/conversations/[conversationId]/messages/route.ts =====
import { NextRequest, NextResponse } from 'next/server';
import prismadb from '@/server/db';
import { MessageContentPart } from '@/lib/types';
import { buildMessagePage, parseMessagePageLimit } from '@/server/utils/messagePage';

type RouteParams = {
  params: Promise<{
    conversationId: string;
  }>;
};

/**
 * Handles GET requests to fetch one page of messages for a specific conversation.
 * Query: `limit` (default 20, max 50), `before` (message id cursor; omitted for the latest page).
 * @param req The Next.js request object.
 * @param params The route parameters, containing `conversationId`.
 * @returns `{ messages, nextCursor }` with messages in chronological order, or an error.
 */
export async function GET(req: NextRequest, { params }: RouteParams) {
  const { conversationId } = await params;
  const clientId = req.headers.get('x-client-id');

  if (!conversationId || !clientId) {
    return NextResponse.json(
```

```
      { error: 'Conversation ID and Client ID are required' },
      { status: 400 }
    );
  }

  const { searchParams } = req.nextUrl;
  const limit = parseMessagePageLimit(searchParams.get('limit'));
  const before = searchParams.get('before');

  try {
    const rowsDesc = await prismadb.message.findMany({
      where: {
        conversationId,
        conversation: { clientId },
      },
      orderBy: [{ createdAt: 'desc' }, { id: 'desc' }],
      take: limit + 1,
      ...(before && { cursor: { id: before }, skip: 1 }),
    });
    return NextResponse.json(buildMessagePage(rowsDesc, limit));
  } catch (error) {
    console.error('[MESSAGES_GET]', error);
    return NextResponse.json({ error: 'Internal Server Error' }, { status: 500 });
  }
}

/**
 * Handles POST requests to add a new message to a conversation.
 * This also implicitly updates the conversation's `updatedAt` timestamp.
 * And triggers a background task to analyze user messages for profile information.
 * @param req The Next.js request object containing the message data.
 * @param params The route parameters, containing `conversationId`.
 * @returns A JSON response with the newly created message or an error.
 */
export async function POST(req: NextRequest, { params }: RouteParams) {
  const { conversationId } = await params;
  const clientId = req.headers.get('x-client-id');

  if (!conversationId || !clientId) {
    return NextResponse.json(
      { error: 'Conversation ID and Client ID are required' },
      { status: 400 }
    );
  }

  try {
    const ownedConversation = await prismadb.conversation.findFirst({
      where: { id: conversationId, clientId },
      select: { id: true },
    });
```

```
    if (!ownedConversation) {
      return NextResponse.json({ error: 'Conversation not found' }, { status: 404 });
    }

    const body = (await req.json()) as { role?: unknown; content?: unknown };
    const { role, content } = body;

    // AI 消息只由服务端生成链路写入（graph/nodes.ts 占位 + helpers.ts 更新），客户端只能写用户消息
    if (role !== 'user' || !Array.isArray(content) || content.length === 0) {
      return NextResponse.json(
        { error: 'Only user messages with non-empty content can be posted' },
        { status: 400 }
      );
    }

    console.log('接收到的消息', JSON.stringify(content))

    const newMessage = await prismadb.message.create({
      data: {
        conversationId: conversationId,
        role: 'user',
        content: content as MessageContentPart[],
        timestamp: new Date(),
      },
    });

    return NextResponse.json(newMessage, { status: 201 });

  } catch (error) {
    console.error('[MESSAGES_POST]', error);
    return NextResponse.json({ error: 'Internal Server Error' }, { status: 500 });
  }
}


// ===== FILE: app/api/conversations/[conversationId]/route.ts =====
import { NextRequest, NextResponse } from 'next/server';
import prismadb from '@/server/db';

// Define the type for the route parameters, accommodating Next.js 15's promise-based params
type RouteParams = {
  params: Promise<{ conversationId: string }>;
};

/**
 * Handles DELETE requests to remove a specific conversation.
 * @param req The Next.js request object (not used in this function).
 * @param params The route parameters, containing the `conversationId`.
 * @returns A JSON response indicating success or failure.
 */
```

```
export async function DELETE(req: NextRequest, { params }: RouteParams) {
  const { conversationId } = await params;
  const clientId = req.headers.get('x-client-id');

  if (!conversationId || !clientId) {
    return NextResponse.json(
      { error: 'Conversation ID and Client ID are required' },
      { status: 400 }
    );
  }

  try {
    const ownedConversation = await prismadb.conversation.findFirst({
      where: { id: conversationId, clientId },
      select: { id: true },
    });
    if (!ownedConversation) {
      return NextResponse.json({ error: 'Conversation not found' }, { status: 404 });
    }

    // Attempt to delete the conversation by its ID.
    // Thanks to `onDelete: Cascade` in the schema, all related messages
    // will be automatically deleted as well.
    await prismadb.conversation.delete({
      where: {
        id: conversationId,
      },
    });

    // Return a success response with no body content.
    return new NextResponse(null, { status: 204 });

  } catch (error) {
    // It's good practice to check for specific Prisma errors,
    // like if the record to delete was not found.
    if (error instanceof Error && error.name === 'P2025') { // Prisma's record not found error code
        return NextResponse.json({ error: 'Conversation not found' }, { status: 404 });
    }

    console.error('[CONVERSATION_DELETE]', error);
    return NextResponse.json({ error: 'Internal Server Error' }, { status: 500 });
  }
}

// ===== FILE: app/api/conversations/route.ts =====
import { NextRequest, NextResponse } from 'next/server';
import prismadb from '@/server/db';
import type { MessageContentPart } from '@/lib/types';

/**
```

```
 * Handles GET requests to fetch conversations for a specific client.
 * Expects a `clientId` as a URL search parameter.
 * @param req The Next.js request object.
 * @returns A JSON response with the list of conversations or an error.
 */
export async function GET(req: NextRequest) {
  const clientId = req.headers.get('X-Client-ID');

  if (!clientId) {
    return NextResponse.json({ error: 'Client ID is required' }, { status: 400 });
  }

  try {
    const conversations = await prismadb.conversation.findMany({
      where: {
        clientId: clientId,
      },
      orderBy: {
        updatedAt: 'desc', // Show most recently updated conversations first
      },
    });
    return NextResponse.json(conversations);
  } catch (error) {
    console.error('[CONVERSATIONS_GET]', error);
    return NextResponse.json({ error: 'Internal Server Error' }, { status: 500 });
  }
}

interface CreateConversationBody {
  title?: unknown;
  firstMessage?: { role?: unknown; content?: unknown };
}

/**
 * Handles POST requests to create a new conversation.
 * Body: `title`, optional `firstMessage` ({ role: 'user', content: MessageContentPart[] })
 * persisted in the same write so a new chat costs one round trip instead of two.
 * @param req The Next.js request object.
 * @returns A JSON response with the newly created conversation or an error.
 */
export async function POST(req: NextRequest) {
  try {
    const body = (await req.json()) as CreateConversationBody;
    const { title, firstMessage } = body;
    const clientId = req.headers.get('X-Client-ID');

    if (!clientId) {
      return NextResponse.json({ error: 'Client ID is required' }, { status: 400 });
    }
```

```
    if (typeof title !== 'string' || !title) {
        return NextResponse.json({ error: 'Title is required' }, { status: 400 });
    }

    if (
      firstMessage !== undefined &&
      (firstMessage.role !== 'user' ||
        !Array.isArray(firstMessage.content) ||
        firstMessage.content.length === 0)
    ) {
      return NextResponse.json(
        { error: 'firstMessage must be a user message with non-empty content' },
        { status: 400 }
      );
    }

    const newConversation = await prismadb.conversation.create({
      data: {
        title,
        clientId,
        ...(firstMessage && {
          messages: {
            create: {
              role: 'user',
              content: firstMessage.content as MessageContentPart[],
              timestamp: new Date(),
            },
          },
        }),
      },
    });

    return NextResponse.json(newConversation, { status: 201 });
  } catch (error) {
    console.error('[CONVERSATIONS_POST]', error);
    return NextResponse.json({ error: 'Internal Server Error' }, { status: 500 });
  }
}

// ===== FILE: app/api/profile/visual-analysis/route.ts =====
import { NextRequest, NextResponse } from 'next/server';
import { Prisma } from '@prisma/client';
import prismadb from '@/server/db';
import { analyzeVisualProfileFromImage } from '@/server/agents/visual/profile';
import { logVisualProfileAudit } from '@/server/logging/visualProfile';
import {
  isClientOwnedUploadUrl,
  mergeVisualAnalysisIntoProfile,
} from '@/server/utils/profileMetadata';
import { profileFromDbRecord } from '@/server/agents/user-profile';
```

```
import { normalizeVisualFeaturesForZh } from '@/lib/hairColorDisplay';

export async function POST(req: NextRequest) {
  const clientId = req.headers.get('X-Client-ID');
  if (!clientId) {
    return NextResponse.json({ error: 'X-Client-ID header is required' }, { status: 400 });
  }

  const startMs = Date.now();

  try {
    const body = await req.json();
    const imageUrl = typeof body?.imageUrl === 'string' ? body.imageUrl.trim() : '';

    if (!imageUrl) {
      return NextResponse.json({ error: 'imageUrl is required' }, { status: 400 });
    }

    if (!isClientOwnedUploadUrl(imageUrl, clientId)) {
      return NextResponse.json({ error: 'Invalid image URL' }, { status: 403 });
    }

    const visualResult = await analyzeVisualProfileFromImage(imageUrl);

    if (!visualResult.is_valid_portrait) {
      void logVisualProfileAudit({
        timestamp: new Date().toISOString(),
        clientId,
        imageUrl,
        durationMs: Date.now() - startMs,
        success: false,
        error: 'not_a_valid_portrait',
      });
      return NextResponse.json(
        { error: '请上传清晰的正面人像照片' },
        { status: 422 }
      );
    }

    const client = await prismadb.clientProfile.upsert({
      where: { id: clientId },
      update: {},
      create: { id: clientId, profileData: {} },
    });

    const existingProfile = (client.profileData as Record<string, unknown>) || {};
    const normalizedVisual = {
      ...visualResult,
      visual_features: normalizeVisualFeaturesForZh(visualResult.visual_features),
    };
```

```
    const mergedProfile = mergeVisualAnalysisIntoProfile(existingProfile, normalizedVisual, imageUrl);

    const updatedClient = await prismadb.clientProfile.update({
      where: { id: clientId },
      data: { profileData: mergedProfile as Prisma.InputJsonValue },
    });

    void logVisualProfileAudit({
      timestamp: new Date().toISOString(),
      clientId,
      imageUrl,
      durationMs: Date.now() - startMs,
      success: true,
      result: {
        skin_tone: visualResult.skin_tone,
        body_shape: visualResult.body_shape,
        hair_color: visualResult.visual_features.hair_color,
      },
    });

    const profileData = profileFromDbRecord(
      (updatedClient.profileData as Record<string, unknown>) || {}
    );

    return NextResponse.json({
      profileData: {
        ...profileData,
        visual_profile_verified: true,
        selfie_image_url: imageUrl,
        selfie_analyzed_at: mergedProfile.selfie_analyzed_at,
        location: typeof mergedProfile.location === 'string' ? mergedProfile.location : undefined,
      },
    });
  } catch (error) {
    const message = error instanceof Error ? error.message : 'Unknown error';
    console.error(`[VISUAL_PROFILE_API] ClientID: ${clientId}`, error);

    void logVisualProfileAudit({
      timestamp: new Date().toISOString(),
      clientId,
      imageUrl: '',
      durationMs: Date.now() - startMs,
      success: false,
      error: message,
    });

    return NextResponse.json({ error: 'Internal Server Error' }, { status: 500 });
  }
}
```

```
// ===== FILE: app/api/upload/route.ts =====
import { NextRequest, NextResponse } from 'next/server';
import { v4 as uuidv4 } from 'uuid';
import { uploadObjectToCOS } from '@/server/services/cos';
import { ensureLlmSafeImageBuffer } from '@/server/utils/image';

const MAX_UPLOAD_BYTES = 10 * 1024 * 1024;

function extensionForMime(mimeType: string): string {
  switch (mimeType) {
    case 'image/png':
      return 'png';
    case 'image/gif':
      return 'gif';
    case 'image/webp':
      return 'webp';
    case 'image/jpeg':
    default:
      return 'jpeg';
  }
}

export async function POST(req: NextRequest) {
  const clientId = req.headers.get('x-client-id');
  if (!clientId) {
    return NextResponse.json({ error: 'Client ID is required' }, { status: 400 });
  }

  const startedAt = Date.now();
  try {
    const formData = await req.formData();
    const file = formData.get('file');
    if (!(file instanceof File)) {
      return NextResponse.json({ error: 'file is required' }, { status: 400 });
    }
    if (!file.type.startsWith('image/')) {
      return NextResponse.json({ error: 'Only image uploads are allowed' }, { status: 400 });
    }
    if (file.size > MAX_UPLOAD_BYTES) {
      return NextResponse.json({ error: 'File too large (max 10MB)' }, { status: 413 });
    }

    const readStartedAt = Date.now();
    const rawBuffer = Buffer.from(await file.arrayBuffer());
    const readMs = Date.now() - readStartedAt;

    const convertStartedAt = Date.now();
    const safe = await ensureLlmSafeImageBuffer(rawBuffer, file.type);
    const convertMs = Date.now() - convertStartedAt;
```

```
    const relativeKey = `user-uploads/${clientId}/${uuidv4()}.${extensionForMime(safe.mimeType)}`;
    const cosStartedAt = Date.now();
    const publicUrl = await uploadObjectToCOS(safe.buffer, safe.mimeType, relativeKey);
    const cosMs = Date.now() - cosStartedAt;

    console.log('[API /api/upload] timing', JSON.stringify({
      outcome: 'success',
      ms: {
        read_file: readMs,
        convert: convertMs,
        cos_upload: cosMs,
        total: Date.now() - startedAt,
      },
      bytes: safe.buffer.length,
      mime: safe.mimeType,
      sourceMime: file.type,
    }));

    return NextResponse.json({ publicUrl });
  } catch (error) {
    console.log('[API /api/upload] timing', JSON.stringify({
      outcome: 'error',
      ms: { total: Date.now() - startedAt },
    }));
    console.error('[API /api/upload]', error);
    return NextResponse.json({ error: 'Internal Server Error' }, { status: 500 });
  }
}


// ===== FILE: app/api/access/logout/route.ts =====
import { NextResponse } from 'next/server';
import { ACCESS_COOKIE_NAME } from '@/server/auth/access';

export async function POST() {
  const response = NextResponse.json({ mode: 'browse' });
  response.cookies.set({
    name: ACCESS_COOKIE_NAME,
    value: '',
    httpOnly: true,
    sameSite: 'strict',
    secure: process.env.NODE_ENV === 'production',
    path: '/',
    maxAge: 0,
  });
  return response;
}
```

```
// ===== FILE: app/api/access/status/route.ts =====
import { NextRequest, NextResponse } from 'next/server';
import {
  ACCESS_COOKIE_NAME,
  isAccessConfigured,
  verifyAccessSessionToken,
} from '@/server/auth/access';
import { getSessionUser } from '@/server/auth/currentUser';
import { getAuthMode } from '@/server/auth/mode';

/** 前端能力探针：access 模式看访问码 cookie，user 模式看登录 session */
export async function GET(req: NextRequest) {
  if (getAuthMode() === 'user') {
    const user = await getSessionUser(req);
    return NextResponse.json({
      mode: user ? 'verified' : 'browse',
      configured: true,
      authMode: 'user',
    });
  }

  const verified = await verifyAccessSessionToken(
    req.cookies.get(ACCESS_COOKIE_NAME)?.value
  );

  return NextResponse.json({
    mode: verified ? 'verified' : 'browse',
    configured: isAccessConfigured(),
    authMode: 'access',
  });
}


// ===== FILE: app/api/access/verify/route.ts =====
import { NextRequest, NextResponse } from 'next/server';
import {
  ACCESS_COOKIE_MAX_AGE_SECONDS,
  ACCESS_COOKIE_NAME,
  createAccessSessionToken,
  isAccessCodeValid,
  isAccessConfigured,
} from '@/server/auth/access';

export async function POST(req: NextRequest) {
  if (!isAccessConfigured()) {
    return NextResponse.json(
      { error: 'Access verification is not configured.' },
      { status: 503 }
    );
  }
```

```
  const body = (await req.json().catch(() => null)) as { code?: unknown } | null;
  const code = typeof body?.code === 'string' ? body.code : '';
  if (!isAccessCodeValid(code)) {
    return NextResponse.json({ error: '访问码不正确' }, { status: 401 });
  }

  const response = NextResponse.json({ mode: 'verified' });
  response.cookies.set({
    name: ACCESS_COOKIE_NAME,
    value: await createAccessSessionToken(),
    httpOnly: true,
    sameSite: 'strict',
    secure: process.env.NODE_ENV === 'production',
    path: '/',
    maxAge: ACCESS_COOKIE_MAX_AGE_SECONDS,
  });
  return response;
}
```